ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

UNIVERSITÀ DEGLI STUDI DI BOLOGNA

SCUOLA DI INGEGNERIA E ARCHITETTURA

Corso di Laurea in Ingegneria Informatica

Dipartimento di Informatica – Scienza e Ingegneria

Tesi di Laurea in Tecnologie Web T

Modelli LiteRT-based per Machine Learning Efficiente su Dispositivi Edge

Tesi di Laurea di Andrea Passini

Relatore:

Prof. Ing. Paolo Bellavista

Anno accademico 2024 – 25

INDICE

Abstract 4

Capitolo 1 – Contesto e Obiettivi 6
1.1 – Obiettivi della Tesi 6
1.2 – Modello di machine-learning 7
1.2.1 – Dati e Interazione con il Modello 7
1.2.2 – Il Training del Modello 8
1.2.3 – Raffinamento del Modello (Refinement Learning) 9
1.2.4 – Preparazione e Interpretazione dei dati: Pre e Post Processing 9
1.2.4 – Inferenza 10
1.3 – Il paradigma dell'AI on the Edge 11
1.3.1 – Differenze tra AI cloud-centrica e AI edge 11
1.3.2 – Privacy e AI edge 13
1.3.3 – Edge AI e sostenibilità 14
1.4 – Verso un'inferenza distribuita moderna 15

Capitolo 2 – LiteRT 16
2.1 – Origini e contesto evolutivo 16
2.1.1 – TensorFlow Lite: la spinta verso il mobile 17
2.1.2 – Limiti di TensorFlow Lite 18
2.2 – Migrazione a LiteRT 20
2.2.1 – Nuovo paradigma: interoperabilità 20
2.2.2 – Retrocompatibilità e sviluppi futuri 21
2.3 – Architettura modulare di LiteRT 21
2.3.1 – Componenti principali 22
2.3.2 – Delegate hardware 23
2.3.3 – Integrazione multi-framework 25
2.4 – Inferenza on-device 27
2.4.1 – Vantaggi dell'inferenza on-device 28
2.4.2 – Sfide dell'inferenza on-device 29
2.5 – Tecniche di ottimizzazione dell'inferenza edge 31
2.5.1 – Quantizzazione 31
2.5.2 – Pruning 33
2.5.3 – Distillazione 35
2.6 – Modelli per il Rilevamento di Anomalie 35
2.6.1 – Cos'è un Autoencoder 35
2.6.2 – Perchè un Autoencoder per l'Anomaly Detection 36
2.7 – Confronto con altre inferenze on-device 37
2.8 – Riflessioni e prospettive.....38

CAPITOLO 3 - Caso di studio 39
3.1 – Panoramica 39
3.1.1 – Tecnologie e Architettura Applicativa 40
3.2 – Object Detection con YOLOv5 41
3.2.1 – Pipeline di conversione 42
3.2.2 – Caricamento del modello e Preprocessing dei dati 44
3.2.3 – Inferenza, Postprocessing e Rendering Visivo 45
3.2.4 – Limiti e Ottimizzazioni 49
3.3 – Anomaly Detection con Autoencoder: Dataset MVTec AD 53
3.3.1 – Preparazione del Dataset 54
3.3.2 – Addestramento dell'Autoencoder 55
3.3.3 – Valutazione delle Prestazioni attraverso Soglie di Rilevamento 56
3.3.4 – Conversione in LiteRT: Inferenza Android 58
3.3.5 – Limiti ed Ottimizzazioni 59
3.4 – Discussione e Commento 62

Conclusioni e Direzioni Future 63

Bibliografia 64

ABSTRACT

La stesura della tesi si basa sull'evoluzione incessante dell'Intelligenza Artificiale rapportata ai dispositivi mobili. Essa è arricchita da un caso di studio di elaborazione locale di due modelli addestrati.

La crescente quantità dei dati generati dai dispositivi *IoT* rende evidente la necessità di infrastrutture di calcolo che superi le barriere intrinseche che caratterizzano i modelli cloud-centrici. Il presente elaborato di tesi esplora il paradigma dell'*AI on the Edge*, una soluzione che sposta le capacità di inferenza direttamente sui dispositivi periferici. Il lavoro si concentra in particolare sull'analisi e la validazione di **LiteRT**, una piattaforma di *runtime* evoluta che rappresenta un'evoluzione strategica nell'approccio di Google all'inferenza distribuita, definita direttamente dagli sviluppatori come *"una libreria edge inference che unifica, semplifica e modernizza l'inferenza AI su dispositivo"*. Dopo un'approfondita disamina del contesto e delle motivazioni che hanno favorito la sua nascita, la stesura ne esplora l'architettura modulare che favorisce l'interoperabilità tra diversi *framework*, mettendo in evidenza l'importanza delle deleghe hardware e delle tecniche di ottimizzazione. L'architettura *on-device* di *LiteRT* affronta sfide complesse gestendo latenza e banda in dispositivi con risorse limitate anche in assenza di connettività.

Per arricchire la trattazione e per tradurre i principi teorici in risultati concreti, il lavoro presenta un caso di studio pratico e completo che utilizza due modelli *LiteRT* distinti. Vengono implementati e valutati due scenari applicativi su dispositivi *Android*: un modello per il rilevamento di oggetti e uno per la rilevazione di anomalie su dataset specifici. L'intero processo, dalla preparazione dei dati e l'addestramento dei modelli fino alla conversione in formato *LiteRT* e l'inferenza *on-device*, è dettagliato per quantificare le prestazioni, i tempi di esecuzione e l'utilizzo delle risorse. In particolare, i risultati ottenuti dimostrano l'efficacia e l'usabilità della tecnologia in contesti, anche complessi, di visione artificiale, consentendo un'inferenza ad alte prestazioni su hardware limitato e mantenendo una precisione adeguata senza richiedere *training* massivi. L'elaborazione locale dei dati consente di identificare potenziali guasti o inefficienze prima che si verifichino, inviando un allarme istantaneo e riducendo drasticamente i tempi di inattività e i costi di manutenzione non pianificata.

In conclusione, questa tesi afferma che l'*AI on the Edge*, supportata da strumenti potenti e flessibili come *LiteRT*, rappresenta un passo fondamentale verso un'intelligenza artificiale più efficiente, sicura e resiliente in tutti gli ambienti.
I risultati del caso di studio sono solamente un piccolo esempio delle potenzialità di questa tecnologia e aprono la strada a immensi utilizzi in settori strategici come la manutenzione predittiva, la logistica automatizzata e il controllo qualità in ambito industriale. L'evoluzione sancisce la transizione verso un'infrastruttura in cui l'*AI on-device* diventerà un componente nativo e indispensabile per l'elaborazione dei dati, rendendo al contempo la distribuzione dei modelli sempre più fluida.

CAPITOLO 1 – Contesto e Obiettivi

Negli ultimi anni, l'intelligenza artificiale ha conosciuto uno sviluppo senza precedenti, estendendosi ben oltre i *data center* e raggiungendo dispositivi che vengono usati quotidianamente, come *smartphone*, *wearable*, elettrodomestici intelligenti e sistemi *IoT*.

Il progressivo spostamento verso un'elaborazione dell'intelligenza artificiale sempre più improntata ai dispositivi periferici, i cosiddetti *device*, porta con sé nuove sfide e numerose opportunità in termini di prestazioni, latenza, privacy dei dati e consumo energetico.

In questo contesto, Google ha sviluppato **LiteRT**, definita direttamente dagli sviluppatori come *"una libreria edge inference che unifica, semplifica e modernizza l'inferenza AI su dispositivo"*. [1.]

Come menzionato sopra, è una nuova libreria di inferenza progettata per facilitare lo sviluppo e l'esecuzione di modelli di machine learning su dispositivi edge in modo efficiente e flessibile. *LiteRT* rappresenta un'evoluzione rispetto a soluzioni precedenti, con un focus su modularità, supporto *multi-framework*, e maggiore compatibilità hardware.

1.1 – Obiettivi della Tesi

L'obiettivo principale di questa stesura è quindi quello di condurre un'analisi approfondita della tecnologia LiteRT, con particolare attenzione alla sua architettura, ai punti di forza e le sfide che la contraddistinguono e alle sue potenziali applicazioni in scenari reali.

Nel perseguire tale obiettivo, la ricerca intende offrire una panoramica teorica del paradigma dell'intelligenza artificiale *edge* soffermandosi in particolar modo sul tema dell'inferenza *on-device*, delineando quindi il contesto tecnologico entro cui *LiteRT* si inserisce.

Saranno esaminate in dettaglio le componenti tecniche e architetturali che costituiscono la libreria, al fine di comprenderne il funzionamento interno e le modalità di interazione

con diversi ambienti di sviluppo e dispositivi hardware. Infine, la trattazione sarà inoltre arricchita da un caso di studio concreto, utile a valutare le prestazioni effettive del sistema in un contesto applicativo reale. Questa ricerca nasce dalla volontà di comprendere a fondo il funzionamento e il potenziale di LiteRT, non solo come tecnologia in senso teorico, ma anche come strumento pratico per sviluppatori e ricercatori nel campo dell'intelligenza artificiale distribuita.

Nel prossimo paragrafo verrà introdotto il concetto di machine-learning, ponendo attenzione all'*AI edge*.

1.2 – Modello di machine-learning

Per comprendere a fondo l'intelligenza artificiale, è utile partire dal suo nucleo operativo: il **modello** di machine learning. Si tratta di una rappresentazione matematica o statistica capace di apprendere dai dati per svolgere compiti predittivi, decisionali o classificatori. È la componente fondamentale dell'intelligenza artificiale: attraverso l'osservazione e l'elaborazione di dati etichettati o non etichettati, il modello è in grado di identificare pattern, correlazioni e strutture sottostanti, generalizzando per affrontare nuovi dati in modo autonomo in futuro.

La crescente disponibilità di dati e potenza computazionale ha permesso una rapida evoluzione di questi modelli, rendendoli centrali in moltissimi ambiti applicativi, sia nel cloud che, come vedremo sempre più spesso, direttamente sui dispositivi *edge*. [2.]

1.2.1 – Dati e Interazione con il Modello

La qualità, la natura e la struttura dei **dati** giocano un ruolo cruciale nel determinare le prestazioni di un modello di machine learning. Il **dataset** costituisce infatti la risorsa primaria attraverso cui il modello apprende, migliora e generalizza il proprio comportamento.

I dati utilizzati nei modelli di *machine learning* possono essere classificati in base alla loro struttura e alla tipologia di informazione che veicolano. Tra questi si distinguono i dati strutturati, come quelli numerici o tabellari, tipicamente organizzati secondo uno schema fisso all'interno di database relazionali. A questi si affiancano i dati visivi,

costituiti da immagini o sequenze di immagini, comunemente impiegati in ambiti quali la visione artificiale. Un'altra categoria rilevante è rappresentata dai dati sequenziali, che includono testi, segnali temporali, dati biometrici e serie storiche, tutti caratterizzati da una dipendenza temporale tra gli elementi. Infine, i dati multimodali combinano informazioni provenienti da fonti eterogenee, come audio e video, o testo e immagini, e sono sempre più utilizzati in applicazioni avanzate che richiedono una comprensione integrata di input complessi.

Per ottenere un apprendimento efficace e attendibile, è essenziale che il dataset sia bilanciato, rappresentativo e di alta qualità. Dataset caratterizzati da rumore, sbilanciamenti di classe, o scarsa rappresentatività possono compromettere il processo di apprendimento e portare a modelli imprecisi o poco generalizzabili e quindi difficilmente utilizzabili in ambienti avanzati, indipendentemente dalla complessità dell'architettura utilizzata. Infine, un'adeguata interazione tra modello e dati implica anche la presenza di meccanismi di validazione e test, fondamentali per monitorare l'evoluzione delle performance.

In ambito *edge*, oltre a porre attenzione alla gestione e alla preparazione dei dati, bisogna tener conto dei vincoli computazionali e di memoria imposti dal dispositivo, rendendo spesso necessaria, come emergerà in seguito, una pre-elaborazione o compressione dei dati prima della fase di inferenza.

1.2.2 Il *Training* del Modello

L'**addestramento** è il processo attraverso cui un modello apprende dai dati, in genere a partire da un dataset etichettato, ovvero un insieme di dati per cui si conosce già l'output corretto, utile per guidare l'apprendimento del modello durante le prime fasi. Il modello analizza questi dati al fine di individuare *pattern*, correlazioni e regolarità utili a formulare previsioni accurate su dati futuri. Senza entrare troppo nel dettaglio, l'obiettivo principale del training è quello di minimizzare l'errore tra le predizioni del modello e i valori reali, ottimizzando i suoi parametri interni. Questo processo avviene in cicli ripetuti, chiamati *epoch*, dove il modello analizza l'intero set di dati di addestramento e si regola per migliorare le sue previsioni, imparando dai propri errori. Ogni *epoch* rappresenta un ciclo completo in cui il modello ha visto e imparato dall'intero set di dati di addestramento. Il modello riesce così ad aggiornare i propri **pesi**,

ovvero i parametri numerici che determinano l'importanza delle varie connessioni tra i neuroni. I pesi sono fondamentali perché influenzano il modo in cui i dati in ingresso vengono trasformati in output, con l'obiettivo finale di minimizzare l'errore di previsione. [4.]

1.2.3 Raffinamento del Modello

Una volta completata la fase iniziale di addestramento, è spesso necessario migliorare ulteriormente le prestazioni del modello attraverso un processo chiamato **raffinamento** (*refinement learning* o *fine-tuning*). Questo avviene tipicamente quando si vuole adattare ulteriormente un modello pre-addestrato a un nuovo compito o a un dominio specifico utile allo sviluppatore, sfruttando le conoscenze già apprese per ridurre i tempi di addestramento e migliorare l'efficienza.

Durante questa fase, il modello viene esposto a un nuovo set di dati, spesso più piccolo ma più mirato, e i suoi parametri vengono ottimizzati in modo più delicato, con un tasso di apprendimento più basso. Questo consente al modello di specializzarsi senza perdere le competenze acquisite precedentemente (*catastrophic forgetting*). [5.]

Anche in questa fase, l'impiego di hardware specializzato, come GPU e TPU, si rivela essenziale soprattutto per dataset di medie dimensioni: tali dispositivi sono progettati per accelerare l'elaborazione di reti neurali profonde, soprattutto appunto quando si lavora con architetture complesse o si effettua un *refinement learning* su modelli di grandi dimensioni.

1.2.4 – Preparazione e Interpretazione dei dati: Pre e Post Processing

I concetti di **preprocessing** e **postprocessing** sono fasi fondamentali della pipeline di machine learning, che vanno oltre la semplice pulizia dei dati. Rappresentano le operazioni strategiche che inquadrano il processo di apprendimento automatico. Esistono numerose tecniche che spaziano e si differiscono molto a seconda di cosa si sta implementando, sono fondamentali in quanto assicurano che il modello riceva informazioni di alta qualità affinché le sue previsioni siano utili e interpretabili.

Il *preprocessing* non è solo un'operazione tecnica, ma una fase cruciale che prepara i dati grezzi per l'algoritmo di apprendimento. Il suo scopo principale è quello di

convertire i dati in un formato che sia più efficiente, informativo e stabile per il modello. In sostanza, si tratta di un'azione preventiva per eliminare le problematiche che potrebbero ostacolare l'apprendimento, come rumore, inconsistenza o squilibrio. [6.]

Il *postprocessing*, al contrario, si applica dopo che il modello ha generato le sue previsioni. La sua funzione teorica non è quella di modificare il modello stesso, ma di gestire e affinare i risultati per renderli più adatti all'applicazione finale e più facilmente interpretabili. Molti modelli di machine learning, infatti, producono output "grezzi" come array di probabilità o coordinate che devono essere convertiti in un formato più significativo per l'utente o per il sistema che li utilizza. [7.]

Entrambe le fasi sono fondamentali per un'applicazione di machine learning di successo. Nel caso di studio della stesura verranno citate particolari tecniche di *processing* per la gestione di immagini nel campo della visione artificiale.

1.2.5 – Inferenza

Dopo l'addestramento, un modello di machine learning entra nella fase operativa. E' qua che viene in soccorso il concetto di **inferenza**, in cui vengono effettuate previsioni o classificazioni su nuovi dati, mai visti prima. A differenza del training, che richiede risorse computazionali elevate e grandi quantità di dati, l'inferenza è un processo più leggero, focalizzato sull'applicazione del modello appreso. [8.]

L'inferenza rappresenta quindi il momento in cui il modello "entra in azione", interpretando gli input e generando gli output in tempo reale. Durante questo processo, il modello elabora i dati di input passando attraverso i vari strati della rete, calcolando in ogni passaggio le **attivazioni**: questi sono i valori intermedi prodotti da ogni neurone dopo aver applicato la funzione di attivazione, e rappresentano la "risposta" del modello agli input in ogni fase della rete.

In ambito *edge*, questo processo deve avvenire con la massima efficienza possibile, spesso su dispositivi con risorse limitate, rendendo fondamentale l'impiego di librerie leggere e ottimizzate.

1.3 – Il paradigma dell'AI on the Edge

Una volta chiariti i principi fondamentali alla base dei modelli di machine learning, risulta opportuno analizzare le modalità attraverso cui tali modelli vengono impiegati nei contesti operativi. In particolare, la recente evoluzione delle tecnologie digitali ha favorito l'emergere di un nuovo paradigma computazionale nel crescente panorama tecnologico.

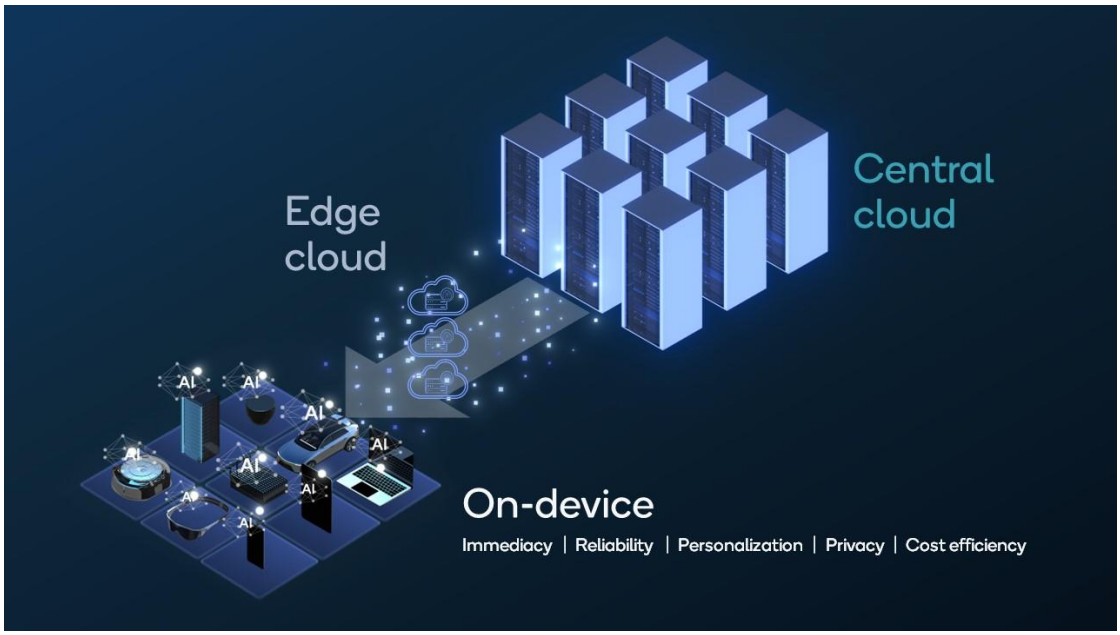
Il paradigma dell'**AI on the Edge** rappresenta un approccio in cui l'intelligenza artificiale viene eseguita direttamente sui dispositivi periferici, *edge devices*, come smartphone, droni, telecamere, sensori *IoT*, automobili, ecc., anziché nei cloud o in *data center* remoti, evitando di dipendere esclusivamente da essi. Rappresenta quindi una risposta concreta a molte delle limitazioni strutturali dell'approccio cloud-centrico, in particolare per quanto riguarda la latenza, la larghezza di banda, l'affidabilità della rete, la gestione della privacy e, non da ultimo, il suo impatto in termini di sostenibilità ambientale. Portare l'intelligenza artificiale più vicino alla fonte del dato consente, infatti, di migliorare significativamente la reattività dei sistemi e di ridurre il rischio associato alla trasmissione di informazioni sensibili a server remoti. [3.]

Questo paradigma è ormai fondamentale in contesti come la *smart mobility*, la videosorveglianza intelligente, la manifattura predittiva, i dispositivi *wearable*, e la robotica autonoma. Tuttavia, presenta anche alcune sfide: la necessità di modelli AI ottimizzati, l'uso di hardware specializzato e la gestione degli aggiornamenti dei modelli distribuiti.

1.3.1 – Differenze tra AI cloud-centrica e AI edge

Il paradigma tradizionale dell'intelligenza artificiale si fonda sul **cloud computing**, un modello in cui i dati vengono raccolti localmente, trasferiti a server remoti per essere elaborati e infine restituiti all'utente sotto forma di output o decisioni. Questo approccio presenta indubbi vantaggi in termini di potenza computazionale, scalabilità e capacità di gestire dataset di grandi dimensioni, rivelandosi particolarmente efficace in scenari centralizzati e ad alta intensità di calcolo. [9.]

Tuttavia, esso mostra importanti limiti in contesti operativi critici come l'automazione industriale, la guida autonoma o la robotica collaborativa, dove la latenza, la disponibilità di rete e la sicurezza dei dati sono fattori determinanti. [10.]



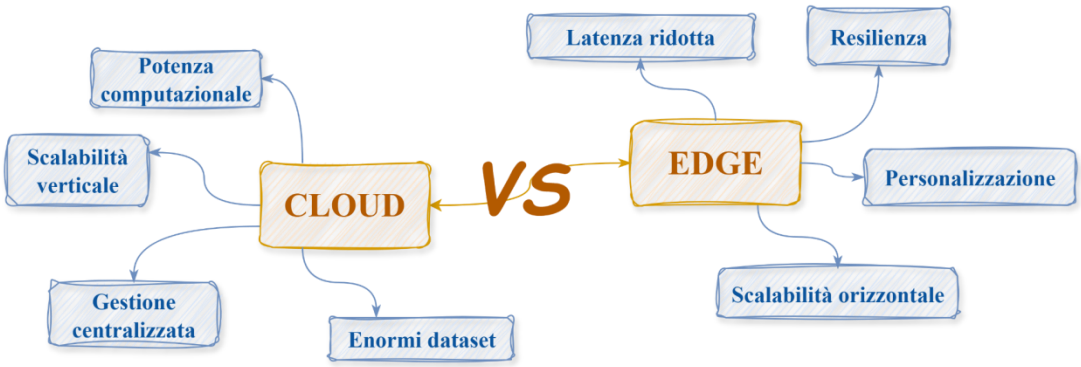
1. Cloud-to-Edge Computing.

In opposizione, l'**AI edge** si dimostra particolarmente adatta a scenari real-time e data-sensitive. Poiché i modelli vengono eseguiti direttamente sul dispositivo vengono ridotti notevolmente i costi e la latenza e viene offerta una maggiore resilienza, consentendo così una risposta più rapida, riducendo notevolmente i costi associati al trasferimento di grandi volumi di dati e garantendo continuità operativa anche in condizioni di connettività limitata o assente. [3.]

I modelli cloud tendono ad essere general-purpose, addestrati su dataset eterogenei e destinati a un pubblico ampio. Al contrario, l'*AI edge* permette la personalizzazione locale del modello, adattandolo al singolo dispositivo o utente grazie a tecniche come il *refinement learning*. Questo approccio rende l'esperienza più contestualizzata e rilevante per l'utente. [11.]

L'aggiornamento e la manutenzione dei modelli basati sul cloud sono uniformi, assicurando una gestione centralizzata dell'intero sistema. Viceversa, nell'ambito edge si presenta il problema della distribuzione efficiente degli aggiornamenti su una vasta rete eterogenea di dispositivi. Tuttavia, tecnologie emergenti come il *Federated*

learning o adottando framework leggeri come *LiteRT* permettono di monitorare e aggiornare i modelli direttamente sui dispositivi. [12.]



2. schema che confronta le soluzioni edge e cloud, sottolineando i punti salienti.

Per concludere, una distinzione importante riguarda il concetto di scalabilità. L'intelligenza artificiale cloud-centrica si presta a una scalabilità verticale, potenziando i *data center*. L'*AI edge*, invece, favorisce una scalabilità orizzontale, sfruttando una rete distribuita di dispositivi intelligenti. Ciò consente di costruire architetture robuste e flessibili, in grado di operare anche in condizioni ambientali o infrastrutturali non ideali. [3.]

1.3.2 – L'esigenza di privacy nell'AI moderna e rapporto con AI edge

La **privacy** rappresenta un elemento centrale nel dibattito sull'intelligenza artificiale moderna, soprattutto alla luce della crescente raccolta di dati sensibili tramite dispositivi intelligenti e sensori sempre attivi. Ogni giorno, *smartphone*, *wearable*, *smart speaker* e dispositivi *IoT* generano informazioni personali, spesso trasmesse a server remoti per l'elaborazione. Dai paragrafi precedenti emerge chiaramente come il modello *cloud-centrico* presenti rilevanti criticità etiche, legali e operative, tra cui il rischio di intercettazione dei dati durante la trasmissione e la difficoltà di rilevare accessi non autorizzati o compromissioni dei server centrali. [13.]

In un contesto così fragile e delicato, l'*Edge AI* rappresenta una soluzione efficace e decentralizzata, poiché consente l'esecuzione dei modelli direttamente sul dispositivo, evitando che i dati sensibili vengano trasmessi a server remoti. Viene garantita una elaborazione sicura e rispettosa della sicurezza e privacy e vengono eliminati quindi i

punti di vulnerabilità legati alla rete e alla memorizzazione centralizzata, rendendo più difficile ogni tentativo di violazione. [14.]

L'*AI on-device*, come quella supportata da librerie leggere come LiteRT, garantisce il massimo livello di protezione dei dati e una maggiore autonomia operativa al sistema.

1.3.3 – Edge AI e sostenibilità

L'adozione dell'intelligenza artificiale on the edge non risponde solo a esigenze tecniche e operative, ma si inserisce anche in una più ampia riflessione sull'impatto ambientale delle tecnologie digitali. I tradizionali modelli di AI cloud-centrica richiedono grandi quantità di energia per l'elaborazione, la memorizzazione e il trasferimento dei dati, aggravati dalla necessità di raffreddamento attivo nei *data center*. Secondo recenti stime, i *data center* globali consumano oltre il 2% dell'energia elettrica mondiale, una percentuale destinata a crescere con la diffusione dell'AI su larga scala. In questo contesto, l'*Edge AI* si configura come una soluzione più sostenibile. Elaborando i dati direttamente sul dispositivo, riduce drasticamente il volume di informazioni da trasmettere e immagazzinare, limitando il traffico di rete e il carico sui server centrali. Inoltre, molti dispositivi *edge* impiegano hardware a basso consumo progettato per ottimizzare l'efficienza energetica. Questa decentralizzazione contribuisce a una gestione energetica più distribuita, scalabile e adattabile al contesto d'uso.

Più in generale, il paradigma *edge computing* può rafforzare concretamente le strategie *ESG, Environmental, Social, and Governance,* delle aziende, integrandosi nei loro obiettivi di sostenibilità. Riducendo la necessità di trasportare enormi quantità di dati verso il cloud, l'*Edge AI* consente di diminuire le emissioni legate al consumo energetico delle infrastrutture *IT*. Inoltre, la sua capacità di abilitare l'analisi locale e in tempo reale dei dati, ad esempio in impianti industriali o sistemi di *smart city*, favorisce un utilizzo più intelligente e ottimizzato delle risorse naturali, riducendo sprechi, consumi superflui e inefficienze. In questa prospettiva, l'*AI* distribuita non è solo un'evoluzione tecnologica, ma parte di un più ampio processo di transizione ecologica e trasformazione digitale sostenibile. [15.]

1.4 – Verso un’inferenza distribuita moderna

Dopo tutte queste considerazioni, si nota rapidamente che, come sempre, la soluzione più efficace, soprattutto per grandi dataset, risiede in una strategia ibrida, in cui cloud ed edge collaborano sinergicamente, sfruttando i punti di forza di ciascun paradigma. L'*Edge AI* non sostituisce il *cloud*, ma si integra ad esso in un'architettura ibrida dove le decisioni più critiche, sensibili e urgenti vengono prese in locale, mentre il *cloud* continua a offrire potenza computazionale per l’addestramento, aggiornamento e l’analisi su larga scala.

3. architettura moderna del cloud e integrazione degli edge-devices con essa.

L’evoluzione delle librerie di inferenza ibride riesce piano piano a colmare il divario tra modelli *AI* complessi e dispositivi periferici. Queste soluzioni offrono strumenti potenti, leggeri e flessibili, compatibili con una varietà di framework e architetture hardware, e sono progettate per fornire un’infrastruttura modulare e scalabile per l’inferenza distribuita.

Nel prossimo capitolo verrà approfondito in particolar modo il nuovo framework di inferenza *on-device LiteRT*.

CAPITOLO 2 – LiteRT

Alla luce delle sfide e opportunità dell'AI on the Edge, questo capitolo analizza **LiteRT**, la sua architettura, le sfide e le opportunità intrinseche che riesce ad offrire.

LiteRT è un *runtime* modulare, ultraleggero e ad alte prestazioni sviluppato da Google e progettato per eseguire modelli ottimizzati di *TensorFlow Lite (.tflite)* provenienti da diversi framework di machine learning. È pensato per l'inferenza on-device in contesti *edge*, caratterizzati da risorse computazionali limitate. In particolare, trova applicazione ideale su microcontrollori, dispositivi *IoT* e smartphone a basso costo, dove latenza ridotta, *footprint* minimale e basso consumo energetico sono requisiti fondamentali. [1.]

2.1 – Origini e contesto evolutivo

Prima di esplorare le caratteristiche e le innovazioni introdotte da LiteRT, è fondamentale comprendere il contesto evolutivo in cui questa tecnologia è emersa, a partire dai due principali framework di machine learning che l'hanno preceduta, *TensorFlow* e *PyTorch*.

Uno dei due protagonisti più influenti è sicuramente **TensorFlow**, un framework di apprendimento automatico sviluppato da Google e rilasciato come progetto open-source nel 2015. Ha raggiunto un'ampia adozione sia in ambito accademico sia industriale grazie al supporto di una vasta gamma di ambienti computazionali, dai laptop ai *data center*, e per sfruttare al meglio qualsiasi tipo di hardware.

L'altro grande protagonista, ossia **PyTorch**, è stato sviluppato da *Facebook AI Research* e rilasciato nel 2016. Ha guadagnato rapidamente popolarità grazie alla sua sintassi intuitiva e al supporto nativo per la definizione dinamica dei modelli. Molto apprezzato dalla comunità di ricerca per la sua facilità d'uso e flessibilità, ha reso più accessibile la sperimentazione di nuovi algoritmi e architetture, ma anch'esso, inizialmente, non era focalizzato sull'ottimizzazione per ambienti a basso consumo. [16.]

4. principali caratteristiche di TensorFlow e PyTorch.

Tuttavia, le prime versioni di questi framework non erano ottimizzate per l'uso in contesti a risorse limitate. Le librerie risultavano pesanti, con dipendenze complesse e un carico computazionale spesso incompatibile con smartphone, dispositivi *IoT* o applicazioni *edge*, dove sono ovviamente fondamentali efficienza energetica e latenza ridotta.

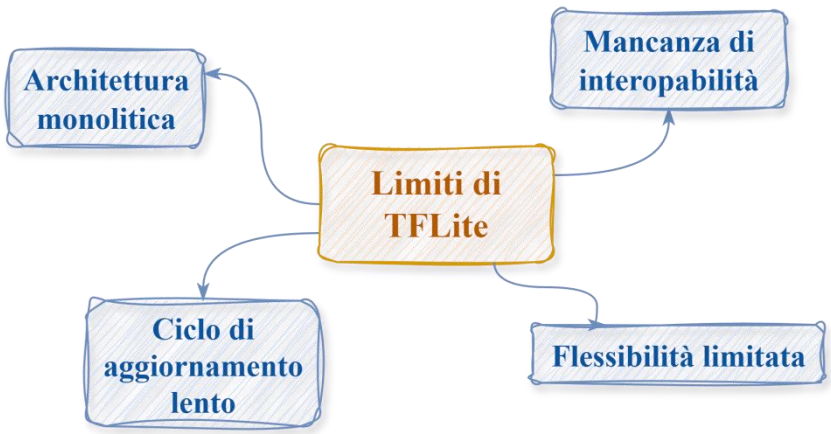
2.1.1 – La spinta verso il mobile - TensorFlow Lite

Con l'espansione delle applicazioni di intelligenza artificiale nei settori mobile, industriale e *IoT*, è emersa in modo sempre più marcato l'esigenza di eseguire inferenze direttamente sui dispositivi *edge*. Questo approccio permette di ridurre la latenza, migliorare la privacy dei dati e garantire una maggiore reattività.
I modelli generati con *TensorFlow* standard risultano inadatti a dispositivi con risorse computazionali limitate, come *CPU* a basso consumo, *RAM* ridotta e alimentazione a batteria. Tali limitazioni hanno portato allo sviluppo di una versione più leggera del framework: **TensorFlow Lite** (*TFLite*).

Lanciato nel 2017, *TensorFlow Lite* è stato progettato espressamente per l'inferenza su dispositivi mobili ed *embedded*, con l'obiettivo di garantire un'implementazione efficiente e a basso consumo energetico. La disponibilità di documentazione, tutorial, esempi pratici e strumenti per la conversione e il deployment dei modelli ha reso in poco tempo *TFLite* uno standard de facto per lo sviluppo di soluzioni *AI* su dispositivi mobili. *TFLite* si contraddistingue per la sua leggerezza, la struttura modulare e la capacità di ridurre la dimensione e la complessità computazionale dei modelli, mantenendo prestazioni adeguate anche in contesti *real-time*. [17.]

2.1.2 – Limiti di TensorFlow Lite

Sebbene *TensorFlow Lite* abbia rappresentato per anni una delle soluzioni più diffuse per l'inferenza di modelli di *machine learning* su dispositivi mobili ed *embedded*, l'evoluzione degli scenari *edge* e l'emergere di esigenze sempre più eterogenee hanno messo in evidenza una serie di limiti strutturali. *TFLite*, infatti, è nato come interprete compatto pensato per eseguire modelli pre-addestrati di *TensorFlow*, ma presenta un'architettura relativamente monolitica.



5. schema che illustra i limiti di TensorFlow Lite.

Questo approccio ha mostrato ben presto i suoi limiti nei contesti *ultra-constrained*, come i microcontrollori e i dispositivi *IoT* a bassissimo consumo, dove la modularità e la possibilità di includere solo i componenti essenziali rappresentano un requisito fondamentale.

Un altro aspetto critico riguarda il supporto limitato a framework esterni. *TensorFlow* Lite è pensato per eseguire modelli nativi di *TensorFlow*, mentre l'integrazione di

modelli sviluppati in ambienti differenti, come *PyTorch* o *ONNX*, richiede complesse operazioni di conversione, spesso soggette a errori, perdita di precisione o mancato supporto per alcune operazioni. Questa mancanza di interoperabilità ha costituito un ostacolo importante alla diffusione di *TFLite* in pipeline di inferenza *multi-framework*, oggi sempre più comuni in ambito industriale e commerciale.

A ciò si aggiunge una ridotta flessibilità rispetto a soluzioni moderne. L'introduzione di delegate personalizzati o l'adattamento di *TFLite* a hardware specializzato risulta spesso complesso, poco documentato e fortemente vincolato all'ecosistema *TensorFlow*. Questo limita la possibilità per gli sviluppatori di sfruttare appieno le potenzialità offerte da acceleratori hardware emergenti, rallentando l'adozione in ambiti dove l'ottimizzazione delle prestazioni è cruciale. [17.]
Anche sul fronte dell'integrazione di pipeline multi-framework, *TFLite* risulta carente. In molti casi reali, infatti, è necessario orchestrare componenti sviluppati in ambienti differenti, ad esempio una parte di rete neurale costruita in *PyTorch* integrata con un modulo di elaborazione in *TensorFlow*. *TFLite* non è stato progettato per gestire nativamente questa coesistenza, risultando poco adatto a scenari complessi e frammentati.
Infine, un ulteriore limite risiede nella gestione degli aggiornamenti e del ciclo di rilascio. Essendo strettamente legato al rilascio generale di *TensorFlow*, *TFLite* non beneficia di un ciclo di sviluppo indipendente o ottimizzato per l'ambiente edge. Questo ha rallentato l'introduzione di innovazioni specifiche per l'inferenza on-device, lasciando irrisolti alcuni problemi legati all'ottimizzazione su hardware di nuova generazione e alla portabilità.

2.2 – Migrazione da TensorFlow Lite a LiteRT: evoluzione di un framework on-device

Con l'evoluzione delle esigenze del machine learning distribuito e l'affermarsi di un'ampia varietà di dispositivi *edge*, le soluzioni precedenti, come sottolineato, hanno mostrato limiti importanti soprattutto dal punto di vista strutturale. **LiteRT** rappresenta l'evoluzione necessaria di *TensorFlow Lite*, progettata per offrire un'inferenza on-device ultraleggera e compatibile con una varietà di framework di machine learning. Questa transizione non solo amplia il supporto a modelli provenienti da diversi ambienti di sviluppo, ma introduce anche un'architettura modulare che facilita l'adattamento a dispositivi *edge* con risorse limitate.

Il 4 settembre 2024, Google ha annunciato un'importante evoluzione del proprio ecosistema di inferenza on-device: *TensorFlow* Lite cambia nome diventando LiteRT. Questo rebranding non rappresenta solo un cambiamento d'identità, ma segna anche un profondo riposizionamento tecnologico. [1.]

Il *repository GitHub di* LiteRT è stato creato per ospitare il nuovo progetto, sebbene inizialmente non fosse destinato allo sviluppo open source, in quanto dipendeva da codice esistente di *TensorFlow* tramite un sottogruppo *Git*. Google ha dichiarato l'intenzione di evolvere il repository affinché gli sviluppatori possano contribuire direttamente in futuro. [18.]

2.2.1 – Il nuovo paradigma, interoperabilità

LiteRT riflette un'evoluzione significativa rispetto al passato, introducendo un approccio agnostico al framework di origine. È progettato per operare in ambienti eterogenei, riducendo la frammentazione dell'ecosistema *AI edge*.

Il suo design promuove la portabilità dei modelli e semplifica il deployment su una varietà di dispositivi, offrendo agli sviluppatori maggiore libertà di scelta rispetto agli strumenti e linguaggi utilizzati. [3.]

2.2.2 – Retrocompatibilità e sviluppi futuri

Nonostante rappresenti una netta evoluzione, questa migrazione verso LiteRT è stata progettata per essere trasparente e graduale. Una delle priorità di Google è infatti garantire la compatibilità con le API esistenti di *TFLite*, semplificando il passaggio per gli sviluppatori già familiari con il framework originario. In pratica, le applicazioni costruite su *TensorFlow* Lite continueranno a funzionare anche con l'introduzione di LiteRT, senza necessità di riscritture complesse o modifiche strutturali.

In linea con la roadmap ufficiale, Google ha chiarito che tutti i futuri sviluppi e aggiornamenti saranno focalizzati su LiteRT, mentre *TensorFlow Lite* verrà mantenuto esclusivamente per assicurare la retrocompatibilità, senza però ricevere ulteriori innovazioni o potenziamenti funzionali. [1.] Questa direzione strategica riflette un chiaro cambio di paradigma nell'approccio di Google all'inferenza on-device. LiteRT rappresenta il cuore evolutivo dell'ecosistema AI distribuito, unificando prestazioni, modularità e interoperabilità in una nuova libreria di riferimento. Il suo design, orientato alla scalabilità e alla compatibilità multi-framework, mira infatti a supportare scenari *edge* sempre più complessi, offrendo agli sviluppatori uno strumento flessibile e pronto per le sfide del futuro.

2.3 – Architettura modulare di LiteRT

In questo paragrafo verrà fornita un'analisi tecnica approfondita dell'architettura di LiteRT, evidenziando come questa tecnologia riesca ad affrontare le sfide tipiche dell'intelligenza artificiale distribuita, precedentemente discusse.

LiteRT adotta o un paradigma **modulare**, con l'obiettivo di offrire un'elevata flessibilità e adattabilità in ambienti *edge*, spesso caratterizzati da risorse limitate e grande eterogeneità hardware. L'architettura si compone di tre elementi fondamentali: *lite_runtime, model_loader e delegate_library*. Ciascun modulo è indipendente e integrabile separatamente, permettendo agli sviluppatori di selezionare solo i componenti realmente necessari. Questo approccio riduce significativamente sia il footprint binario che l'impiego di risorse a *run-time*. [1.]

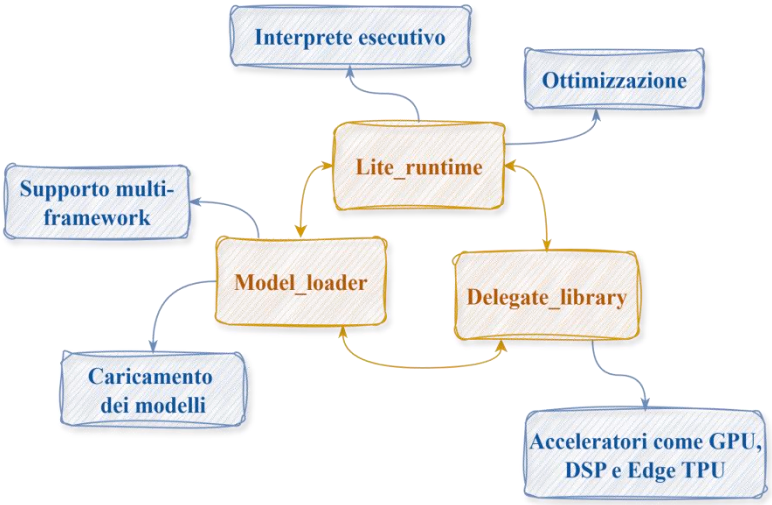
La struttura modulare di LiteRT si rivela particolarmente efficace nei contesti embedded e *IoT* industriali, dove coesistono pipeline *AI* eterogenee, basate su modelli provenienti da framework differenti e destinate a dispositivi con capacità computazionali molto variabili. LiteRT permette così una personalizzazione granulare dell'ambiente di inferenza, adattabile a un'ampia gamma di scenari operativi.

2.3.1 – Componenti principali dell'architettura LiteRT:

I componenti principali dell'architettura di LiteRT sono progettati con l'intento di operare in modo sinergico, ma possiedono anche un elevato grado di indipendenza, il che consente di integrare ciascuno di essi in maniera separata a seconda delle necessità.

I tre componenti fondamentali sono:

- **Lite Runtime (o interpreter core)**: costituisce il nucleo esecutivo del sistema. È responsabile dell'inferenza dei modelli di machine learning, garantendo un footprint minimo e massima efficienza computazionale. La sua architettura minimalista non solo ottimizza l'utilizzo delle risorse di calcolo, ma consente anche di eseguire inferenze in tempo reale, essenziali in contesti industriali o applicazioni che richiedono risposte rapide e continue operando anche in ambienti che pongono vincoli di memoria, internet o potenza computazionale.
- **Model Loader:** gestisce il caricamento dei modelli, consentendo una rapida inizializzazione e migliorando le prestazioni generali delle operazioni di inferenza, anche in scenari real-time. È progettato per leggere modelli in formati binari ottimizzati, riducendo significativamente i tempi di avvio. Inoltre, si distingue per la sua capacità di gestire diversi formati di modelli provenienti da vari framework, permettendo una facile integrazione in contesti multi-framework, riducendo i tempi di conversione e aumentando la portabilità dei modelli attraverso differenti piattaforme.
- **Delegate Library (o accelerator)**: si occupa della mappatura dinamica dell'inferenza su hardware specializzato, come GPU, NPU, TPU o DSP, sfruttando acceleratori disponibili nel dispositivo. Consente quindi di scalare e ottimizzare l'inferenza in modo preciso, adattandola alle caratteristiche e risorse specifiche dell'hardware disponibile. [19.]



6. panoramica per i componenti di LiteRT.

Questa architettura flessibile minimizza i tempi di inizializzazione e garantisce un utilizzo più efficiente delle risorse disponibili, affermando LiteRT come una piattaforma run-time capace di rispondere in modo granulare alle esigenze del moderno computing distribuito on-device. [1.]

2.3.2 – Delegate hardware

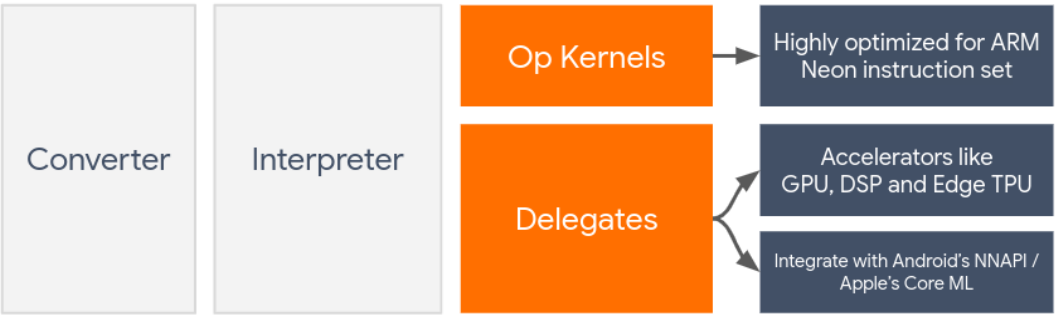
L'hardware gioca un ruolo cruciale nell'ottimizzazione delle performance e nell'efficienza delle inferenze nei dispositivi edge.
Il concetto di ***delegation hardware*** è fondamentale nell'Edge AI: consiste nel spostare il carico computazionale dalla CPU a unità hardware specializzate, ottenendo inferenze più rapide, efficienti e con minor consumo energetico.
Il supporto ai delegate hardware costituisce uno degli elementi più rilevanti e strategici dell'evoluzione architetturale che ha portato alla nascita di LiteRT.

A differenza di TensorFlow Lite, che soffre di vincoli architetturali fortemente legati all'ecosistema TensorFlow, LiteRT adottando una struttura modulare consente l'integrazione semplificata di delegate personalizzati o nativi, progettati proprio per sfruttare appieno le caratteristiche di piattaforme hardware specializzate.

A livello pratico, ciò avviene tramite le **API dei Delegates**, che consentono a LiteRT di "delegare" l'esecuzione di specifiche operazioni all'acceleratore hardware. Esempi concreti di questi delegati includono il *GPU Delegate* (*TFLiteGpuDelegate*) e il *Neural*

Networks API (NNAPI) Delegate per l'ambiente Android, che reindirizzano l'inferenza per ottenere un'efficienza superiore. [20.]

Nel caso di studio verranno approfondite queste API in ambienti *Android* nel paragrafo sulle ottimizzazioni. Tra le piattaforme supportate troviamo Edge TPU (Google), NPU, DSP e GPU. Questi acceleratori, grazie alla loro capacità di eseguire operazioni tensoriali in parallelo con efficienza energetica, risultano fondamentali per garantire inferenze rapide, stabili e a basso consumo su dispositivi edge. [19.]



7. panoramica della guida di LiteRT su acceleratori hardware.

Ogni tipo di delegate è pensato per ottimizzare uno specifico profilo computazionale:

- Le **GPU** (*Graphics Processing Unit*), sono particolarmente adatte per eseguire operazioni di calcolo parallelo su larga scala, ideali per compiti ad alta intensità computazionale come il deep learning.
- Le **NPU** (*Neural Processing Unit*), invece, sono ottimizzate per accelerare specificamente le operazioni di inferenza di reti neurali, migliorando significativamente l'efficienza energetica e riducendo i tempi di inferenza.
- I **DSP** (*Digital Signal Processor*), eccellono nell'elaborazione di segnali, particolarmente utili in applicazioni dove la latenza è molto bassa e la precisione è cruciale.
- Infine, le **TPU** (*Tensor Processing Unit*) di Google, sono progettate per accelerare l'elaborazione di operazioni tensoriali, ottimizzando i carichi di lavoro, con significativi miglioramenti nelle prestazioni e nella velocità di inferenza.

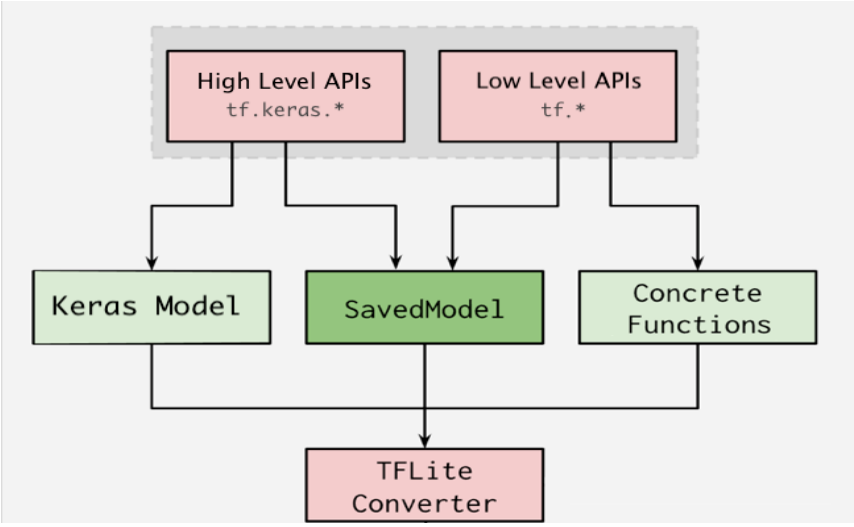
Per concludere, la capacità di adattarsi dinamicamente all'hardware disponibile consente a LiteRT di sfruttare appieno le caratteristiche specifiche di ciascuna piattaforma, superando così i limiti imposti dagli approcci monolitici tradizionali.

2.3.3 – Integrazione multi-framework: compatibilità tecnica e formati supportati

Come è stato precedentemente accennato, uno degli aspetti distintivi di LiteRT è la sua capacità di operare in scenari **multi-framework**. Questo approccio segue una logica *"hub-and-spoke"* dove i modelli convertiti in *.tflite* rappresentano il nodo centrale. LiteRT consente di personalizzare facilmente l'ambiente di inferenza in base alle esigenze specifiche dell'applicazione. In particolare, offre vantaggi significativi in scenari *multi-framework* e *multi-device*, riducendo costi e tempi di integrazione, migliorando efficienza e portabilità delle soluzioni.

LiteRT è progettato per eseguire modelli provenienti da diversi framework di machine learning, tra cui *TensorFlow*, *PyTorch*, *JAX*, *Keras* e *ONNX*, seguendo appunto un paradigma basato sull'interoperabilità e sulla flessibilità. A rendere possibile questa eterogeneità contribuisce in maniera decisiva il componente *model_loader*, incaricato di caricare i modelli ottimizzati in un formato compatibile con l'ambiente runtime, riducendo di molto i costi di integrazione.

Google ha inoltre pubblicato una guida tecnica che dettaglia gli operatori supportati da LiteRT, distinguendo tra quelli disponibili nativamente, quelli che richiedono delegate o estensioni, e quelli da evitare per massimizzare l'efficienza su dispositivi edge. Ciò risulta particolarmente importante nella fase di conversione di modelli da altri framework verso *.tflite*. [21.]



8. pipeline di conversione TFLiteConverter.

Ecco quali modelli sono supportati per la conversione e in che modo:

- **TensorFlow / Keras**: La compatibilità è nativa poiché fanno parte dello stesso ecosistema sviluppato da Google. I modelli possono essere convertiti efficacemente utilizzando la **TensorFlow Lite Converter API**. Questa fondamentale API utilizza funzioni come la *tf.lite.TFLiteConverter.from_keras_model()* che accettano in input modelli esportati in formato *SavedModel*, ossia il formato standard e ufficiale di serializzazione di TensorFlow per l'inferenza e il deployment. Grazie a tutto ciò si riesce a garantire il massimo supporto agli operatori anche applicando ottimizzazioni notevoli durante il processo e l'integrazione è perfetta. [22.]

```
converter = tf.lite.TFLiteConverter.from_keras_model(autoencoder)
```

- **ONNX** *(Open Neural Network Exchange)*: sebbene LiteRT non supporti ONNX nativamente, è possibile convertire i modelli *.onnx* in *.tflite* attraverso passaggi intermedi usando strumenti di conversione disponibili come *onnx-tf*, *tf2onnx* o pipeline customizzate
- **PyTorch**: LiteRT non supporta nativamente *PyTorch*, ma è possibile usare una pipeline di conversione esportando prima il modello pytorch in formato *.onnx* usando *torch.onnx.export* e solo dopo convertendolo in un modello *.tflite*. Attualmente solo un sottoinsieme degli operatori *ONNX* è compatibile e durante la pipeline possono essere necessarie delle modifiche manuali per mappare operatori non supportati nativamente. Durante il processo è importante semplificare il modello prima della conversione.
- **Modelli personalizzati**: LiteRT offre anche la possibilità di integrare modelli custom, a scapito delle dimensioni e della portabilità, e spesso solo attraverso *delegate hardware* o modifiche custom al *runtime*.

Alla luce di ciò, l'approccio multi-framework consente quindi agli sviluppatori di utilizzare la tecnologia con la quale sono più familiari, senza compromettere le prestazioni o la portabilità del modello. Esso promuove, di fatto, un comportamento agnostico rispetto all'ambiente di origine, consentendo allo sviluppatore di operare con modelli sviluppati in contesti diversi senza dipendere da uno specifico framework. [21.]

2.4 – L’inferenza on-device

L’inferenza può avvenire anche su dispositivi con capacità limitate, come smartphone, microcontrollori, dispositivi *IoT* o *embedded system*. Essa deve essere progettata per essere estremamente rapida ed efficiente, in modo da garantire prestazioni ottimali anche su dispositivi con capacità computazionali ridotte. La sfida, quindi, non è solo quella di applicare il modello, ma anche farlo in tempi brevi e con un basso consumo energetico.

L'**inferenza on-device**, ovvero l'esecuzione del modello direttamente sui dispositivi periferici senza necessità di passare attraverso il server *cloud*, ha guadagnato un'importanza crescente negli ultimi anni. Le inferenze possono avvenire anche in contesti di connettività intermittente, dove l'assenza di una connessione di rete stabile non pregiudica il funzionamento del sistema. [23.]

L’esecuzione del modello ormai convertito è gestita in python dal *TensorFlow Lite Interpreter API*, dove l'interprete carica e gestisce il modello, utilizzando classi e metodi chiave come *tf.lite.Interpreter* per la creazione dell'interprete, e successivamente *interpreter.invoke()* per l'esecuzione effettiva del calcolo sul dispositivo. Questa API è utile per sviluppo e conversioni su desktop, in ambienti come *Python*.

Quando si tratta di modelli LiteRT, spesso è utile fare inferenza direttamente nell’applicazione edge. In questo contesto, le *API Android in Java/Kotlin*, che sfruttano l’oggetto *Interpreter*, risultano fondamentali. Questo oggetto è il cardine dell'inferenza on-device, poiché è responsabile di caricare il modello e gestire la sua esecuzione. Nel corso del prossimo capitolo, verranno approfondite le tematiche relative all’inferenza e verranno mostrati due esempi concreti che illustrano l'uso dell'interprete, come evidenziato dalla riga di codice, la quale crea l’interprete *TensorFlow Lite*.

```
// Crea l'interprete TensorFlow Lite e assegna il delegato GPU
tflite = new Interpreter(loadModelFile(assetManager, modelPath));
```

Come prima, l'interprete, una volta creato, è pronto per l'esecuzione, ma l'inferenza vera e propria viene avviata solo successivamente, attraverso il metodo *tflite.run()*. L'interprete funge da intermediario tra il tuo codice e il modello, gestendo l'allocazione della memoria e l'esecuzione dei calcoli. [17.]

2.4.1 – Vantaggi dell’inferenza on-device

L’inferenza on-device offre una serie di vantaggi strategici rispetto all’inferenza tradizionale basata su cloud, rendendola una scelta preferibile in numerosi scenari applicativi, specialmente nell’ambito dell’*Edge AI*. I principali benefici includono:

- **Riduzione della latenza**: poiché i dati non devono essere inviati a server remoti per l'elaborazione, l'inferenza avviene in tempo reale direttamente sul dispositivo. Questo è fondamentale in applicazioni dove il tempo di risposta è cruciale, come nei veicoli autonomi, nei sistemi di controllo industriale o nella realtà aumentata.
- **Scalabilità e costi operativi ridotti:** eseguire una parte significativa dell'elaborazione localmente diminuisce la dipendenza dalle infrastrutture cloud e allevia il carico sulla rete. Questo si traduce in costi operativi più contenuti, sia per quanto riguarda il traffico dati che le infrastrutture server necessarie per supportare l'inferenza centralizzata.
- **Maggiore privacy e sicurezza dei dati**: eseguendo l'inferenza direttamente in locale, i dati sensibili rimangono al sicuro, evitando così il rischio di violazioni della privacy o attacchi durante il trasferimento. Questo metodo è particolarmente cruciale in settori come la sanità, la videosorveglianza e le applicazioni consumer che gestiscono informazioni personali.
- **Maggiore reattività e personalizzazione**: l’inferenza *on-device* permette risposte rapide e adattate al contesto, facilitando l’implementazione di sistemi intelligenti che si adattano in tempo reale all’ambiente circostante o alle preferenze dell’utente, senza necessità di attendere l’elaborazione in remoto.
- **Indipendenza dalla connettività**: in molti contesti operativi, come quelli remoti, sui dispositivi mobili o nelle infrastrutture industriali, può capitare che la connessione a internet sia assente, instabile o costosa. L'inferenza *on-device* assicura che le operazioni possano continuare anche quando la connettività è intermittente o completamente assente. [11.]

Queste caratteristiche rendono l’*edge inference* una soluzione ideale per applicazioni mobili, domotica e smart home, diagnostica medica portatile, veicoli autonomi, robotica decentralizzata e sistemi industriali, dove l’elaborazione in tempo reale consente il

monitoraggio predittivo, il controllo di qualità automatizzato e la manutenzione preventiva senza dover ricorrere a connessioni cloud continue.

2.4.2 – Sfide dell'inferenza on-device

Sebbene l'inferenza *edge* offra numerosi vantaggi, presenta anche una serie di sfide e limitazioni che devono essere giustamente affrontate per garantire il suo successo. Queste limitazioni riguardano sia le risorse hardware che le difficoltà nel mantenere alta la qualità delle inferenze in ambienti con capacità computazionale e memoria limitate. Le principali sfide includono inevitabilmente:

- **Consumo energetico**: il consumo energetico rappresenta la principale criticità in questo contesto. Molti dispositivi *edge* operano con batterie o sono soggetti a limitazioni energetiche, il che rende difficile eseguire inferenze con modelli complessi senza compromettere l'autonomia. I processi di inferenza richiedono una notevole potenza di calcolo e, quindi, una quantità significativa di energia. Per le applicazioni mobili o portatili, la sfida è rendere l'inferenza efficiente dal punto di vista energetico, minimizzando il consumo senza sacrificare troppo le performance.
- **Risorse computazionali limitate**: la maggior parte dei dispositivi *edge*, come smartphone, dispositivi *IoT* e microcontrollori, ha risorse hardware piuttosto limitate rispetto ai server dei *data center*. In particolare, la potenza di calcolo e la memoria rappresentano vincoli critici. I modelli di machine learning, sebbene siano molto potenti, necessitano di una notevole quantità di risorse computazionali durante l'inferenza. Senza hardware specializzato come le *NPU* o acceleratori hardware dedicati, l'inferenza di modelli complessi risulta lenta e inefficiente, rischiando di non soddisfare i requisiti delle applicazioni in tempo reale.
- **Aggiornamenti e deployment di modelli**: è stato sottolineato come l'elaborazione locale favorisca la scalabilità; è altrettanto importante evidenziare che questo comporta delle difficoltà legate agli aggiornamenti continui dei modelli. Nei sistemi *edge*, i modelli devono essere aggiornati periodicamente per adattarsi a nuovi dati e migliorare le prestazioni. Tuttavia, l'aggiornamento dei modelli su dispositivi con risorse limitate può essere un processo complesso. I modelli addestrati su server cloud potrebbero non essere facilmente trasferibili

e adattabili ai dispositivi *edge*, e la necessità di ridurre le dimensioni dei modelli rende ancora più difficile mantenere il processo di aggiornamento agile ed efficiente. Inoltre, gli aggiornamenti frequenti possono aumentare il carico di rete, il che potrebbe essere problematico in scenari con connettività intermittente.

- **Complessità multi-dispositivo**: come è già stato ampiamente discusso, i dispositivi *edge* variano notevolmente in termini di potenza di calcolo, memoria e architettura hardware. Una soluzione di inferenza che funziona bene su un tipo di dispositivo purtroppo non sempre potrebbe funzionare altrettanto efficacemente su un altro. Gestire un'inferenza che funzioni in modo ottimale su diversi dispositivi, da smartphone a microcontrollori a *SoC* avanzati, è una sfida significativa. LiteRT, con la sua architettura modulare, cerca di affrontare questa sfida, ma la compatibilità con hardware diversificato rimane un aspetto critico e delicato da gestire.
- **Robustezza e affidabilità nei contesti reali**: l'inferenza *edge* deve essere in grado di funzionare in ambienti variabili e talvolta difficili, come veicoli autonomi o dispositivi IoT remoti. Le condizioni operative possono cambiare rapidamente, e il sistema di inferenza deve essere abbastanza robusto da rispondere in modo affidabile a situazioni inaspettate. La presenza di rumore nei dati, interferenze nei segnali, e altre variabili impreviste possono influenzare la qualità dell'inferenza, richiedendo tecniche avanzate di gestione dei dati e correzione degli errori. [10.]

2.5 – Tecniche di ottimizzazione dell'inferenza edge

Per affrontare al meglio le sfide e rendere l'inferenza on-device davvero praticabile e scalabile, è necessario ricorrere a una serie di tecniche di ottimizzazione dei modelli pensate specificamente per questo contesto. Queste strategie hanno l'obiettivo di ridurre la complessità computazionale, diminuire la dimensione del modello e abbassare i consumi energetici, mantenendo al contempo una buona accuratezza predittiva.

In questo contesto si inserisce **LiteRT**. Esso è pensato per facilitare e standardizzare l'inferenza ottimizzata su *edge* utilizzando un ecosistema che integra strumenti per la quantizzazione, il *pruning* e molte altre tecniche, contribuendo quindi in modo concreto all'evoluzione dell'AI distribuita.

Questo paragrafo esplora nel dettaglio le più importanti strategie oggi utilizzate di ottimizzazione, adottabili in ambienti *edge*, evidenziandone le funzionalità, i vantaggi e i casi d'uso più comuni, con un focus particolare sul supporto e i vantaggi offerti da LiteRT.

2.5.1 – Quantizzazione

La **quantizzazione** è diventata una delle tecniche più efficaci e diffuse per migliorare le prestazioni e ridurre notevolmente le dimensioni dei modelli di *deep learning* destinati all'inferenza su dispositivi dove le risorse hardware sono vincolate. Consiste nel ridurre la precisione numerica con cui sono rappresentati i pesi e le attivazioni del modello. Invece di utilizzare il formato a *floating point 32-bit*, tipico dei modelli *full precision*, si passa a rappresentazioni più compatte, come interi a 8 bit *(int8)* o *floating point 16-bit*. Questa operazione riduce significativamente la dimensione del modello e migliora la velocità di inferenza, specialmente su hardware che supporta nativamente operazioni a bassa precisione. LiteRT supporta diverse modalità di quantizzazione post-addestramento, che si differenziano per il grado di compressione e per i requisiti di calibrazione. Queste ottimizzazioni sono gestite tramite il **TensorFlow Model Optimization Toolkit**. [17.] Per la quantizzazione, si utilizzano funzioni dell'API di conversione precedentemente mostrate, impostando *converter.optimizations* a *default*.

```
# Quantization - API: TensorFlow Model Optimization Toolkit
converter.optimization = [tf.lite.Optimize.DEFAULT]
```

Di seguito le principali modalità supportate:

- **Quantizzazione float16:** questa modalità converte i pesi a *floating point 16 bit*, mantenendo le attivazioni in *floating point 32 bit*. È particolarmente utile su dispositivi dotati di GPU o acceleratori che supportano nativamente *floating point 16 bit*. Offre un buon compromesso tra riduzione della dimensione del modello e mantenimento dell'accuratezza, con una riduzione contenuta rispetto alla quantizzazione a interi. Sarà proprio quella che verrà utilizzata nel caso di studio.

  La funzione *convert_to_tflite_fp16()* in figura 9, che sarà approfondita nel caso di studio, offre un esempio concreto di come il *TFLiteConverter* venga istruito per applicare queste ottimizzazioni.

```python
def convert_to_tflite_fp16():
    # Converte il SavedModel in TFLite con quantizzazione FP16
    saved_dir = "yolov5s_saved_model"

    # TensorFlow Lite Converter API.
    converter = tf.lite.TFLiteConverter.from_saved_model(saved_dir)

    # Quantization - API TensorFlow Model Optimization Toolkit
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.target_spec.supported_types = [tf.float16]
    tflite_model = converter.convert()

    tflite_file_path = "yolov5s-fp16.tflite"
    with open(tflite_file_path, "wb") as f:
        f.write(tflite_model)

    print(f"Modello TFLite FP16 creato: '{tflite_file_path}'")
    return tflite_file_path
```

9. funzione convert_to_tflite_fp16().

- **Quantizzazione a intervallo dinamico** *(Dynamic Range Quantization)*: in questa modalità, i pesi del modello sono convertiti a 8 bit, mentre le attivazioni rimangono in *floating point*. Questo approccio è semplice da applicare e comporta una riduzione della dimensione del modello fino a circa quattro volte, con un miglioramento della velocità di inferenza, soprattutto su *CPU* e hardware che supportano operazioni intere. La precisione del modello generalmente rimane elevata.

- **Quantizzazione a interi** *(Full Integer Quantization)***:** In questo caso, sia i pesi che le attivazioni sono quantizzati a 8 bit. LiteRT necessita di un dataset di calibrazione, un campione rappresentativo dei dati di input, per stimare la distribuzione delle attivazioni e determinare i parametri di quantizzazione. Questo metodo sfrutta al massimo gli acceleratori hardware, riducendo ulteriormente dimensione e latenza. [24.]

Come abbiamo visto, LiteRT fornisce ottimi strumenti di analisi e monitoraggio degli errori di quantizzazione, permettendo agli sviluppatori di valutare l'impatto della compressione sulle prestazioni del modello in modo dettagliato. Una pratica comune è confrontare l'output di un modello a precisione completa con quello di un modello quantizzato su un dataset di validazione, analizzando i risultati strato per strato.

Per concludere, qualsiasi variante della quantizzazione permette di ottenere modelli compatti, veloci e meno energivori, rendendo possibile l'*AI on-device* in scenari reali e complessi.

2.5.2 – Pruning

Il **pruning** è una tecnica di ottimizzazione che consiste nella "potatura" di connessioni o neuroni meno significativi all'interno di una rete neurale, con l'obiettivo di ridurre la complessità del modello senza compromettere significativamente le sue prestazioni. In pratica, durante o dopo l'addestramento, vengono individuati i pesi con valori prossimi a zero o con scarso contributo all'output, che possono essere rimossi o azzerati. Questo alleggerisce la rete, diminuisce la quantità di memoria necessaria per memorizzare i pesi, e riduce i calcoli richiesti durante l'inferenza.
Anche questa operazione è particolarmente vantaggiosa in scenari *edge*. In LiteRT, il *pruning* può essere integrato nella pipeline di ottimizzazione attraverso strumenti compatibili con il *TensorFlow Model Optimization Toolkit (TF-MOT)*. Questo approccio consente di applicare tecniche di potatura dei pesi durante o dopo l'addestramento, prima della conversione del modello in formato *.tflite*, rendendolo adatto all'inferenza su dispositivi *edge*. Grazie a questa integrazione, LiteRT può eseguire modelli potati e ottimizzati in modo efficiente, ottenendo riduzioni significative in termini di dimensioni e latenza, senza compromettere sensibilmente la qualità delle predizioni.

TF-MOT mette a disposizione diverse modalità di *pruning*, tra cui:

- **Pruning basato sulla magnitudine dei pesi**: esso rimuove iterativamente i pesi con valori assoluti più piccoli durante l'addestramento. Questa strategia incrementa la *sparsity* del modello, riducendo il numero di parametri attivi. LiteRT supporta questa modalità e consente di sfruttarne i benefici anche su dispositivi a risorse limitate, mantenendo alta l'efficienza del modello quantizzato.
- **Pruning strutturato:** durante questo processo vengono rimosse intere unità strutturali del modello, rendendo il modello finale più adatto a hardware che beneficia di matrici dense. Questa tecnica è ideale per l'inferenza su acceleratori *edge*, in quanto riduce la complessità computazionale mantenendo l'allineamento con gli ottimizzatori hardware di LiteRT.
- **Pruning consapevole dell'addestramento** *(Training-aware Pruning)*: integra il pruning nel processo di training stesso, permettendo al modello di adattarsi gradualmente alle modifiche strutturali. Questa forma di *structured fine-tuning* aumenta la robustezza e stabilizza la perdita di accuratezza dovuta alla potatura. LiteRT favorisce questo approccio offrendo compatibilità piena con i modelli ottenuti tramite *TF-MOT* e semplificando l'adozione in pipeline già esistenti.

In sintesi, il *pruning* non solo riduce la dimensione del modello e i requisiti di memoria, ma può anche migliorare la velocità di inferenza, specialmente su dispositivi che supportano operazioni sparse consentendo agli sviluppatori di progettare modelli sempre più leggeri e flessibili. Tuttavia, è sempre necessario eseguire una fase di *fine-tuning* dopo il *pruning* per recuperare eventuali perdite di accuratezza e ottimizzare ulteriormente le prestazioni del modello. [25.]

L'utilizzo combinato di *pruning* e quantizzazione rappresenta ad oggi una delle strategie più efficaci per l'ottimizzazione *edge-ready* dei modelli di *deep learning*.

2.5.3 – Distillazione del modello

La **distillazione del modello** è una tecnica che consente di ridurre la complessità di un modello senza una significativa perdita di performance, trasferendo la conoscenza da un modello grande e complesso, *teacher*, a un modello più piccolo, veloce e meno complesso, *student*. Durante questo processo, il modello studente apprende le rappresentazioni e le caratteristiche rilevanti dal modello insegnante, ma utilizzando un numero significativamente inferiore di parametri. In questo modo, si ottiene una notevole riduzione del *footprint* computazionale, senza compromettere le prestazioni e l'accuratezza del modello.

LiteRT sfrutta questa tecnica per creare velocemente modelli leggeri, che possono essere eseguiti più efficacemente sui dispositivi mantenendo la qualità delle previsioni. Questo è particolarmente utile in applicazioni che richiedono risposte rapide *real-time*.

2.6 – Modelli per il Rilevamento di Anomalie

Dopo aver esplorato le tecniche di ottimizzazione per l'inferenza on-device, è necessario arricchire la stesura con esempi di architetture di modelli di *anomaly detection* che, per loro stessa natura, sono particolarmente adatte e probanti a questi contesti.

2.6.1 – Autoencoder

Tra queste architetture spicca l'**autoencoder**. Queste nozioni torneranno utili durante il capitolo tre dove viene analizzato nel dettaglio il caso di studio; la pipeline di *anomaly detection* utilizza un modello *autoencoder*. Rappresenta un esempio perfetto di modello che, grazie alla sua architettura intrinsecamente leggera e focalizzata, si presta magnificamente all'inferenza su dispositivi *edge*, in particolare per compiti come il rilevamento di anomalie.

Un *autoencoder* è una rete neurale progettata per imparare in modo non supervisionato una rappresentazione compressa dei dati. È un modello che cerca di replicare il proprio input nell'output. Questo approccio apparentemente semplice consente alla rete di apprendere le caratteristiche essenziali dei dati. L'obiettivo principale non diventa quello di classificare, ma di apprendere la "normalità" dei dati su cui viene addestrato, in modo da poter identificare successivamente tutto ciò che devia dalla normalità. Il

modello viene addestrato per minimizzare l'errore di ricostruzione, cioè la differenza tra l'input originale e la sua ricostruzione. L'errore di ricostruzione è tipicamente misurato tramite metriche come l'Errore Quadratico Medio, *MSE*, e serve come segnale per aggiornare i pesi della rete durante l'addestramento.

L'addestramento si esegue in genere attraverso la funzione *"fit"*, tipica delle librerie come *Keras*, che automatizza il processo di ottimizzazione per minimizzare la funzione di perdita. [26.]

```
# 3. Addestramento del Modello
print("Inizio addestramento del modello...")
history = autoencoder.fit(X_train_normal, X_train_normal,
                          epochs=50,
                          batch_size=32,
                          shuffle=True,
                          validation_split=0.1) # Usa il 10% dei dati per la validazione
print("Addestramento completato.")
```

10. codice che simula l'addestramento dell'autoencoder usando la funzione ausiliaria "fit".

2.6.2 – Perché un Autoencoder per l'Anomaly Detection

L'efficacia dell' *autoencoder* nel rilevamento di anomalie deriva dalla sua natura di apprendimento. Quando il modello viene addestrato esclusivamente su immagini non anomale, "normali", apprende a ricostruire queste immagini con un errore di ricostruzione molto basso. Quando l'*autoencoder* si trova di fronte a un'immagine che contiene un'anomalia, l'errore di ricostruzione è significativamente più alto. Questo perché la rete non ha mai visto questo tipo di "deviazione" durante l'addestramento e quindi non sa come ricostruirla correttamente.

La differenza nell'errore di ricostruzione tra dati normali e anomali è la chiave per la classificazione: se l'errore supera una soglia predefinita, l'immagine viene etichettata come anomala. La difficoltà diventa quindi quella di stabilire la soglia, la scelta di una soglia più bassa aumenterà la sensibilità del modello rilevando un numero superiore di anomalie, causando però un inevitabile aumento di falsi positivi. Al contrario, una soglia più alta ridurrà i falsi positivi, ma potrebbe causare la mancata identificazione di alcune anomalie. [27.]

2.7 – Confronto con altre inferenze on-device

Per comprendere ancora meglio, il valore e il posizionamento di LiteRT nel panorama dell'*Edge AI*, è fondamentale confrontarlo con altre soluzioni di inferenza on-device. È già stato descritto nel paragrafo relativo al supporto multi-framework come LiteRT garantisce il supporto ad esse, favorendo il *cross-platform*. Adesso verranno esaminate nel dettaglio le differenze con le principali alternative.

PyTorch Mobile, ora inglobato in *PyTorch Edge*, è la risposta di *Facebook AI Research* alla necessità di eseguire modelli *PyTorch* su dispositivi mobili e *edge*. Offre un *runtime* ottimizzato per la *CPU*, *GPU* e *DSP*, e supporta l'esportazione di modelli in formati ottimizzati come *TorchScript*. [28.]

Garantisce un'integrazione nativa con l'ecosistema *PyTorch* e limita le condivisioni *multi-framwork*, offrendo una curva di apprendimento ridotta per gli sviluppatori che già operano in tale ambiente. La sua flessibilità è un punto di forza, grazie al grafo dinamico di *PyTorch*. Spesso, purtroppo, può talvolta presentare un *footprint* maggiore rispetto a LiteRT per modelli simili, il che non è un limite da sottovalutare. Inoltre, il supporto a hardware accelerato può richiedere integrazioni più complesse rispetto ai delegati di LiteRT.

ONNX Runtime è un *runtime* di inferenza ad alte prestazioni per modelli *ONNX*, *Open Neural Network Exchange*. Offre un'eccellente interoperabilità tra framework e supporta una vasta gamma di hardware e sistemi operativi. Purtroppo è piuttosto vincolante in quanto richiede forzatamente la conversione del modello nel formato *ONNX*; il processo può risultare complesso per operatori non standard. Per garantire compatibilità, come già detto, LiteRT può importare modelli da ONNX tramite una fase di conversione. Inoltre, anche qua, a livello di *footprint* binario ed efficienza energetica, specialmente su microcontrollori di dispositivi *edge* a bassissimo consumo, le prestazioni di *ONNX Runtime* tendono a non eguagliare quelle di LiteRT. [29.]

Core ML è il framework proprietario di Apple per l'integrazione di modelli di machine learning nelle applicazioni per i sistemi operativi *iOS* e *macOS*. La sua principale forza risiede nella profonda integrazione con l'ecosistema Apple e nell'ottimizzazione per

l'hardware nativo, inclusi i *Neural Engine*, garantendo prestazioni di inferenza notevolmente elevate sui dispositivi supportati. [30.]

Tuttavia, queste qualità comportano un notevole svantaggio: la sua natura proprietaria e la dipendenza dall'ambiente Apple lo rendono una soluzione non *cross-platform*. Inoltre, il suo sviluppo è legato esclusivamente ad Apple, e l'assenza di un modello *open-source* limita la sua evoluzione da parte della comunità di sviluppatori esterni.

2.8 – Riflessioni e Prospettive

Questo capitolo ha esplorato a fondo il *run-time LiteRT*, posizionandolo come una risposta evoluta e necessaria alle sfide e opportunità dell'*AI on the Edge*. E' stato tracciato dettagliatamente il suo percorso, segnando un cambiamento di paradigma nell'approccio di Google all'inferenza distribuita. È stata esaminata la tecnologia, riaffermando i principi e le motivazioni alla base dell'inferenza on-device. Sono stati messi in luce i suoi molteplici benefici, senza tralasciare le sfide intrinseche associate ad essa. È quindi scontato ribadire come LiteRT non sia solamente un semplice rebranding di *TensorFlow Lite*, bensì una vera e propria evoluzione tecnologica che mira a unificare e semplificare lo sviluppo e il deployment dell'intelligenza artificiale sui dispositivi. Questa direzione strategica è destinata a posizionare LiteRT come uno strumento di riferimento per le future applicazioni Android in ambienti distribuiti, promettendo di sbloccare un potenziale ancora più ampio per il *machine learning* all'avanguardia.

Nel prossimo capitolo l'attenzione si sposterà dai principi teorici alla valutazione pratica attraverso un caso di studio concreto. Verranno analizzate nel dettaglio le prestazioni dei modelli scelti, i tempi di inferenza e l'utilizzo delle risorse per quantificare l'idoneità di LiteRT in scenari reali, sottolineando eventuali limitazioni intrinseche e strutturali di contesti complessi e distribuiti, che cercheranno di essere colmate attraverso ottimizzazioni mirate.

CAPITOLO 3 - Caso di studio

Come ampiamente discusso nel Capitolo teorico, l'adozione di un'architettura modulare come quella di LiteRT e l'utilizzo di tecniche di ottimizzazione sono essenziali per il successo dell'inferenza *on-device*. Questo approccio non solo risponde a esigenze critiche come la protezione della privacy, eliminando la dipendenza dal *cloud*, ma garantisce anche una bassa latenza, fondamentale per le applicazioni che richiedono risposte in tempo reale.

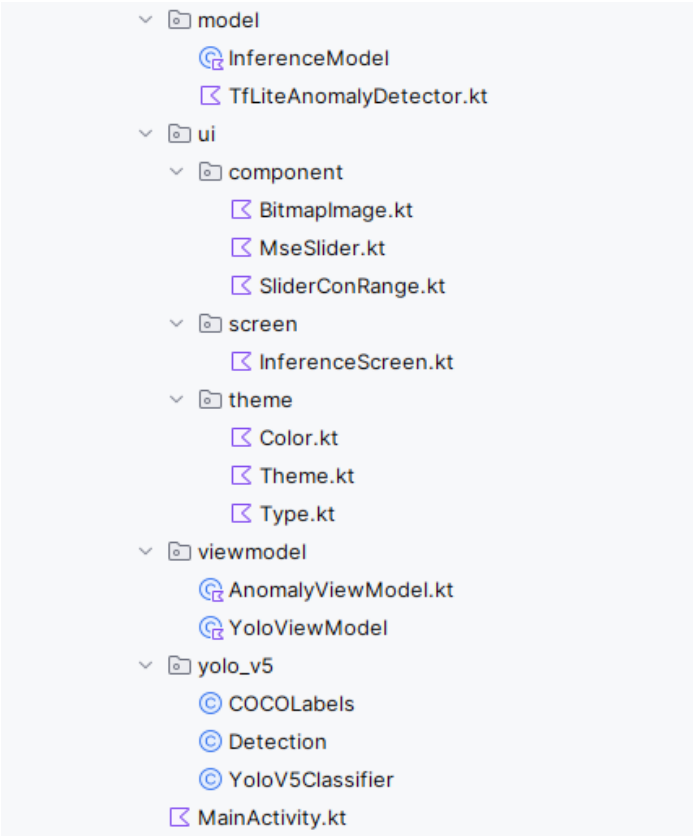
3.1 – Panoramica

L'obiettivo principale di questo capitolo è dimostrare l'efficacia e l'utilità di *LiteRT* per l'inferenza su dispositivi mobili in scenari reali, come quello del caso di studio che verrà analizzato.
Per fare ciò, verrà illustrato un flusso di lavoro completo e riproducibile: dalla preparazione e ottimizzazione dei modelli di *machine learning* fino alla loro integrazione finale in un'applicazione Android moderna.
Questo approccio pratico fornisce al lettore una comprensione tangibile della tecnologia, trasformando i principi teorici in un'applicazione utile per il settore industriale. Il capitolo analizza le sfide e le difficoltà ricorrenti, evidenziando al contempo i numerosi vantaggi di tale implementazione.

Il caso di studio presenta un sistema di visione artificiale *on-device* per Android che elabora immagini della galleria o della fotocamera.
L'applicazione in questione integra sia un detector, ossia un classificatore di immagini, che si basa su un modello *YOLOv5 pytorch* pre-addestrato reperito on-line, sia un modello di *anomaly detection* addestrato tramite uno script *python*. Entrambi i modelli sono convertiti in formato *.tflite* attraverso la pipeline di conversione. L'applicazione visualizza i risultati direttamente sull'immagine e offre pannelli di statistiche e soglie configurabili dall'utente.

3.1.1 – Tecnologie e Architettura Applicativa

Le tecnologie che sono state utilizzate durante l'implementazione dell'applicazione android sono molteplici. L'applicazione stessa è stata scritta nel linguaggio di programmazione *Kotlin*, con l'ausilio di qualche classe *Java*. Sono state utilizzate le componenti di *Jetpack Compose Material*, con packaging ottimizzato per dimensione e compatibilità, che rendono l'applicazione più scalare e flessibile, nonché esteticamente apprezzabile.



11. Organizzazione dei package dentro all'ambiente android studio.

L'architettura segue il pattern *Model-View-ViewModel (MVVM)*, dove i *ViewModel* e le *Coroutines* gestiscono la logica di business e le operazioni asincrone, mentre le schermate dell'interfaccia utente si concentrano esclusivamente sulla presentazione. I permessi necessari nell'*AndroidManifest.xml* includono l'accesso alla fotocamera e alla memoria esterna per l'acquisizione e la gestione delle immagini.

L'app è strutturata con una *UI* principale, *InferenceScreen*, che gestisce il caricamento delle immagini e offre slider per la configurazione delle soglie di entrambi i modelli. La logica applicativa è incapsulata in due ViewModel dedicati alle due pipeline ed

entrambi i modelli sono eseguiti localmente tramite l'*Interpreter TFLite*, con preprocessing e postprocessing dedicati. [17.]

Le responsabilità sono separate: il preprocessing prepara i dati per l'inferenza, l'inferenza esegue il modello, e il postprocessing interpreta e visualizza i risultati. Tutte le fasi sono allineate al codice di implementazione, e sono cruciali per entrambi i modelli.

L'applicazione utilizza il profiler di Android Studio per l'ottimizzazione e il controllo delle prestazioni, supportando input da galleria e fotocamera e fornendo statistiche post-inferenza tramite finestre di dialogo. Il sistema registra eventi chiave di caricamento modelli, trasformazioni, inferenze e salvataggi. Gli errori sono gestiti nei *ViewModel* con messaggi coerenti verso l'*UI*.

Durante la stesura dei prossimi paragrafi verranno illustrati tutti i procedimenti citati nel dettaglio, documentando il caso di studio attraverso le due pipeline distinte. Tale separazione è di natura prettamente stilistica e funzionale, pensata per illustrare in modo chiaro e sequenziale le scelte progettuali, i compromessi e i risultati. Tuttavia, è fondamentale sottolineare che il filo conduttore che unisce entrambi i percorsi è l'adozione e la dimostrazione dell'efficacia di *LiteRT*, il vero argomento cardine di questa trattazione, che rende possibile l'implementazione di soluzioni di machine learning efficienti su dispositivi edge.

3.2 – Object Detection con YOLOv5

Per il rilevamento degli oggetti in tempo reale è stato impiegato un modello *YOLOv5s* pre-addestrato sul vasto dataset *COCO*, una risorsa standard del settore che consente di riconoscere ottanta classi di oggetti comuni. [32.]

Il modello è stato ottimizzato per l'inferenza on-device, salvato con pesi a virgola mobile a 16 bit e configurato per accettare un input/output a 32 bit.
La gestione delle classi e il mapping delle etichette sono conformi allo standard *COCO* e vengono gestiti internamente all'applicazione, garantendo coerenza e accuratezza nella presentazione dei risultati.

3.2.1 – Pipeline di conversione

Un unico script Python automatizza l'intero processo di conversione di *YOLOv5*, partendo dal modello *PyTorch YOLOv5* e arrivando a un file *.tflite* ottimizzato per dispositivi mobili. Questo approccio riproduce in modo concreto la pipeline sequenziale discussa nel capitolo precedente, garantendo un processo efficiente e riproducibile. Ecco le funzioni fondamentali:

- **Preparazione dell'ambiente**: il primo passo verifica l'esistenza della cartella *yolov5*. Se non è presente, lo script si occupa di clonare il repository ufficiale di *ultralytics* da *GitHub*. Successivamente, si sposta in quella cartella e utilizza *pip* per installare tutte le dipendenze richieste dal progetto.

```python
def setup_environment():
    #Clona il repository YOLOv5 e installa le dipendenze.
    print("Preparazione dell'ambiente...")
    if not os.path.exists("yolov5"):
        run_command(["git", "clone", "https://github.com/ultralytics/yolov5.git"])

    os.chdir("yolov5")

    run_command([sys.executable, "-m", "pip", "install", "-r", "requirements.txt"])
    run_command([sys.executable, "-m", "pip", "install", "tensorflow-cpu==2.20.0"])
```

12. Funzione che prepara l'ambiente, clona il repository e installa le dipendenze.

- ***Download* dei pesi del modello**: viene controllato se il file dei pesi pre-addestrati è già presente. Se non lo è, lo scarica direttamente dall'URL github ufficiale. [32.]
- **Esportazione in *SavedModel***: L'architettura *PyTorch* viene convertita in un formato intermedio, il TensorFlow *SavedModel*. Questa conversione è gestita dallo script *export.py* fornito dagli sviluppatori di YOLOv5. Tale passaggio è fondamentale, poiché prepara il modello per la successiva e definitiva conversione nel formato *TFLite*. [33.]

```
Export a YOLOv5 PyTorch model to other formats. TensorFlow exports authored by https://github.com/zldrobit.

Format                  | `export.py --include`         | Model
---                     | ---                           | ---
PyTorch                 | -                             | yolov5s.pt
TorchScript             | `torchscript`                 | yolov5s.torchscript
ONNX                    | `onnx`                        | yolov5s.onnx
OpenVINO                | `openvino`                    | yolov5s_openvino_model/
TensorRT                | `engine`                      | yolov5s.engine
CoreML                  | `coreml`                      | yolov5s.mlmodel
TensorFlow SavedModel   | `saved_model`                 | yolov5s_saved_model/
TensorFlow GraphDef     | `pb`                          | yolov5s.pb
TensorFlow Lite         | `tflite`                      | yolov5s.tflite
TensorFlow Edge TPU     | `edgetpu`                     | yolov5s_edgetpu.tflite
TensorFlow.js           | `tfjs`                        | yolov5s_web_model/
PaddlePaddle            | `paddle`                      | yolov5s_paddle_model/
```

13. la sezione in cui è presente lo script export.py del github di Yolov5.

- **Conversione in TFLite *(FP16)***: Questa fase rappresenta concretamente il cuore della pipeline di ottimizzazione e utilizza concretamente le API fondamentali introdotte teoricamente nello scorso capitolo. [34.]
  Il modello *SavedModel* viene caricato e convertito utilizzando il *TFLiteConverter* di TensorFlow. Per aumentare le prestazioni on-device, lo script applica le ottimizzazioni di default, attraverso, *tf.lite.Optimize.DEFAULT* e, in particolare, la quantizzazione a virgola mobile a 16 bit. [17.]

```python
def convert_to_tflite_fp16():
    # Converte il SavedModel in TFLite con quantizzazione FP16
    saved_dir = "yolov5s_saved_model"

    # TensorFlow Lite Converter API.
    converter = tf.lite.TFLiteConverter.from_saved_model(saved_dir)

    # Quantization - API TensorFlow Model Optimization Toolkit
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    converter.target_spec.supported_types = [tf.float16]
    tflite_model = converter.convert()

    tflite_file_path = "yolov5s-fp16.tflite"
    with open(tflite_file_path, "wb") as f:
        f.write(tflite_model)

    print(f"Modello TFLite FP16 creato: '{tflite_file_path}'")
    return tflite_file_path
```

14. funzione che converte in formato SavedModel con le ottimizzazioni a default.

Questa tecnica riduce significativamente la dimensione del file del modello e accelera l'inferenza, pur mantenendo un'accuratezza elevata.
Il file ottimizzato viene quindi salvato come *yolov5s-fp16.tflite* nella stessa cartella.

In sintesi, richiamando le funzioni sopra citate, si riescono a gestire in modo efficiente tutti i passaggi necessari alla conversione *multi-framework* senza richiedere l'interazione manuale dello sviluppatore tra i vari passaggi.

3.2.2 – Caricamento del modello e Preprocessing dei dati

Per garantire la massima efficienza e minimizzare la latenza, il modello *LiteRT* viene caricato nella cartella *assets* dell'applicazione Android tramite la mappatura in memoria *MappedByteBuffer*. Questo approccio consente un accesso diretto ai dati del modello, eliminando i ritardi associati all'I/O tradizionale.

L'input richiesto dal modello è un tensore di forma *[1, 640, 640, 3]*, che rappresenta un'immagine RGB con risoluzione *640x640 pixel*. I valori dei pixel sono in formato a virgola mobile a 32 bit e sono normalizzati nell'intervallo *[0, 1]*. L'output del modello è un tensore di forma *[1, 25200, 85]*, dove ogni riga contiene le informazioni relative a una potenziale rilevazione.

Il **pre-processing** dei dati di input è una fase critica che precede l'inferenza. Il modulo *YoloV5Classifier* si occupa di questa operazione, che include:

- **Ridimensionamento**: Il bitmap di input viene ridimensionato a 640x640 pixel senza l'uso di *letterboxing*, il che può causare una leggera distorsione ma garantisce un'inferenza più rapida.
- **Normalizzazione**: I valori dei canali RGB di ciascun pixel vengono estratti e normalizzati dividendo per 255.0, scalando i valori nell'intervallo *[0, 1]*.
- **Impacchettamento**: I dati normalizzati vengono impacchettati in un *ByteBuffer* diretto con ordine nativo per massimizzare la velocità di trasferimento dei dati all'interprete *TFLite*. Questo processo garantisce che l'input sia perfettamente allineato alle specifiche del modello.

3.2.3 – Inferenza, Postprocessing e Rendering Visivo

Nel capitolo teorico è stato già ampiamente discusso il ruolo cruciale dell'*Interpreter TensorFlow Lite* nel processo di inferenza. Il componente è l'intermediario tra il codice dell'applicazione e il modello, gestendo l'allocazione della memoria e l'esecuzione dei calcoli di basso livello.

In questo contesto, la funzione *loadModelFile*, in figura 15, si occupa di caricare il modello *LiteRT* dal file system dell'app in una mappatura di memoria.

```
// Carica il file .tflite dal file system
private MappedByteBuffer loadModelFile(AssetManager assetManager, String modelPath) throws IOException {
    AssetFileDescriptor fileDescriptor = assetManager.openFd(modelPath);
    FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
    FileChannel fileChannel = inputStream.getChannel();
    long startOffset = fileDescriptor.getStartOffset();
    long declaredLength = fileDescriptor.getDeclaredLength();
    return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength);
}
```

15. funzione che si occupa del caricamento del modello nell'ambiente di sviluppo.

Successivamente, l'interprete viene istanziato utilizzando la *loadModelFile(assetManager, modelPath))*, che lo rende pronto a ricevere e processare i dati. [17.]

Lo scopo della stesura, in questo punto, si sposta sulle funzioni astratte create per adattarsi al caso di studio. La fase di inferenza e il successivo postprocessing dei dati sono orchestrati tramite funzioni chiave, *runInference(), detect()* e *drawBoxes()*, che lavorano in sinergia per produrre il risultato finale visivo. Queste funzioni sono definite con il modificatore *suspend*, il che le rende ideali per le operazioni asincrone. Grazie all'utilizzo di *viewModelScope.launch*, i codici vengono eseguiti su un thread background separato dalla UI, evitando blocchi onerosi dell'interfaccia utente durante il calcolo intensivo dell'inferenza.

La funzione *runInference()*, in figura 16, gestita da *YoloViewModel*, è il punto di ingresso per l'intero processo di rilevamento e delega il compito principale al modello di inferenza, invocando la funzione *detect()* cuore pulsante della fase di inferenza.

```
fun runInference(onFinished: (Bitmap?) -> Unit) {  1 Usage
    viewModelScope.launch { this: CoroutineScope
        checkInference = false
        val result: Pair<Bitmap, List<Detection>>? = withContext( context = Dispatchers.Default) { this: CoroutineScope
            // Esegui l'inferenza e ottieni il bitmap annotato e la lista di rilevazioni
            inferenceModel.detect(selectedBitmap,  detectionThreshold = threshold,  classifier = classifier!!)
        }
```

16. funzione runInference nel viewmodel delega alla detect l'inferenza.

Il tensore grezzo è rappresentato dall'array *float[][][] output*, che viene popolato dal metodo *tflite.run()*. La sua struttura non è una semplice lista di riquadri, ma un complesso insieme di dati non filtrati e non ancora elaborati. [17.]

```
// Ridimensiona l'immagine a 640x640
Bitmap resizedBitmap = Bitmap.createScaledBitmap(bitmap, INPUT_SIZE, INPUT_SIZE,  filter: true);

// Converte il bitmap in input ByteBuffer (formato float32 RGB normalizzato [0,1])
ByteBuffer inputBuffer = convertBitmapToByteBuffer(resizedBitmap);

// Alloca array di output della dimensione corretta per YOLOv5
float[][][] output = new float[1][OUTPUT_ROWS][OUTPUT_COLUMNS];

// Esegue inferenza
tflite.run(inputBuffer, output);
```

17. porzione di codice che gestisce il bitmap ed esegue inferenza sul tensore grezzo.

Dentro *detect()* viene chiamata la funzione privata *drawBoxes()*, come è possibile notare nella figura 18, che postprocessa e renderizza l'output grezzo ottenuto, rappresentando il fulcro della rilevazione. Questo passaggio è fondamentale per rendere usabile l'output del modello.

```
drawBoxes(
    bitmap = selectedBitmap.copy( config = Bitmap.Config.ARGB_8888,  isMutable = true),
    outputs = output,
    detectionThreshold
) ^withContext
```

18. Chiamata alla funzione drawboxes, il viewmodel delega il rendering dell'immagine.

La funzione *drawBoxes()* ha, quindi, prima di tutto il compito cruciale di interpretare le previsioni grezze, filtrando per ottenere un insieme pulito e significativo di rilevazioni. Tale processo si articola in due passaggi tecnici fondamentali che fanno da filtro per le rilevazioni.

Il primo è l'applicazione di una soglia di confidenza sull'*objectness*. Ogni potenziale rilevazione del modello è associata a un valore che ne indica la probabilità di contenere un oggetto. Il metodo filtra le previsioni applicando una soglia regolabile dall'interfaccia

utente, con un *range* piuttosto dinamico da *0,4* a *0,7*, che scarta in modo efficiente tutte le previsioni con una bassa probabilità. L'impostazione di questa soglia offre un ottimo compromesso tra precisione e falsi positivi: una soglia più alta aumenta la precisione ma rischia di perdere rilevazioni corrette con bassa confidenza. L'integrazione di un cursore nella *user interface* consente all'utente di effettuare test e di ottimizzare questa soglia in base alle proprie esigenze.

Il secondo passaggio è la *Soppressione Non Massima (NMS)*, un algoritmo che risolve il problema delle rilevazioni ridondanti per lo stesso oggetto. L'algoritmo calcola l'*Intersection over Union (IoU)* per ogni coppia di riquadri sovrapposti. [35.] Se l'*IoU* supera la soglia predefinita del 70%, il riquadro con la confidenza più bassa viene scartato in favore di quello con la confidenza più alta. Questa logica è implementata nel metodo *applyNMS()*, in figura 19, della classe ausiliaria *Detection.java*, che ordina e filtra iterativamente le rilevazioni sovrapposte o duplicate per garantire che ogni oggetto sia rappresentato da un unico riquadro ottimale.
Una volta che le rilevazioni sono state filtrate, il metodo si occupa anche del *rendering* visivo, come suggerito dal nome stesso.

```
// Funzione per applicare il Non-Maximum Suppression (NMS)
public static List<Detection> applyNMS(List<Detection> detections, float iouThreshold) {  1 usage
    // Ordinare per confidenza
    Collections.sort(detections, ( Detection d1,  Detection d2) -> Float.compare(d2.objectness, d1.objectness));

    List<Detection> filteredDetections = new ArrayList<>();

    while (!detections.isEmpty()) {
        // Prendi la previsione con la confidenza più alta
        Detection bestDetection = detections.remove( index: 0);
        filteredDetections.add(bestDetection);

        // Filtra le detections che si sovrappongono troppo (IoU > soglia)
        detections.removeIf( Detection detection -> computeIoU(bestDetection, detection) > iouThreshold);
    }

    return filteredDetections;
}
```

19. funzione cardine per il rendering, evita la creazione di rettangoli sovrapposti.

Le coordinate normalizzate dei riquadri vengono scalate rispetto alle dimensioni originali dell'immagine. Infine, i rettangoli di rilevamento e le etichette delle classi *COCO*, con le relative percentuali di confidenza, vengono disegnati sull'immagine attraverso la funzione *drawRect* di *canvas*, fornendo il risultato finale all'utente.

```
canvas.drawRect(left, top, right, bottom, boxPaint)

val className = COCOLabels.getLabel( classId = detection.classId)
val confidence = String.format(Locale.US,  format = "%.2f",  ...args = detection.objectness)
canvas.drawText( text = "$className ($confidence)",  x = left,  y = top + 60f, textPaint)
```

20. codice che utilizza la drawRect per disegnare effettivamente le rilevazioni.

In sintesi, l'architettura separa chiaramente le responsabilità: il *ViewModel* gestisce lo stato e l'orchestrazione del flusso di lavoro in background, mentre il modello di inferenza si occupa delle operazioni computazionali intensive, garantendo una *UI* fluida e reattiva.

Una volta che il procedimento di inferenza e postprocessing dell'immagine è completato, *runInference()* gestisce il risultato. Se il modello restituisce un esito positivo, aggiorna la lista di oggetti rilevati, il *bitmap* visualizzato con i riquadri di rilevamento disegnati e notifica alla UI che il processo è concluso.

Il risultato finale dell'inferenza viene presentato all'utente attraverso un report rapido visualizzato in una finestra di dialogo, che mostra una lista degli oggetti rilevati ordinati per confidenza. Vengono disegnati sull'immagine i rettangoli delle rilevazioni che hanno superato con successo entrambi i due passaggi tecnici fondamentali, la soglia e l'NMS. In figura 21, è mostrato l'esempio in cui il modello riesce effettivamente a classificare una persona e una bici, reperendo le corrette etichette; entrambe le rilevazioni hanno superato i controlli imposti. Inoltre, per una maggiore utilità, viene offerta l'opzione per salvare l'immagine annotata in formato *.png* nella galleria del dispositivo.

21. esempio di risultato di rendering dopo l'inferenza.

3.2.4 – Limiti e Ottimizzazioni

La soluzione attuale, pur essendo funzionale, presenta delle limitazioni intrinseche che ne condizionano l'efficacia e le prestazioni.

Il limite più evidente della soluzione risiede nella **precisione di rilevazione**. Nonostante *YOLOv5* sia un modello all'avanguardia pre-addestrato su un dataset esteso come *COCO*, non sempre vengono rilevati tutti gli oggetti possibili. Questa variabilità è intrinseca ai modelli di apprendimento profondo, che, per quanto robusti, possono mostrare limitazioni nella generalizzazione a contesti non perfettamente allineati al dataset di addestramento originale o in presenza di oggetti di piccole dimensioni, occlusioni o condizioni di illuminazione difficili. [36.]
Il modello, pur essendo altamente performante, manifesta quindi delle limitazioni strutturali che ne compromettono le prestazioni in scenari specifici, rendendo la sua accuratezza non sempre perfetta.

Inoltre, la soluzione manifesta delle **limitazioni strutturali** che ne compromettono le prestazioni. L'algoritmo di *Non-Maximum Suppression (NMS)* implementato è di tipo single-class e utilizza un valore di intersezione fisso al 70%, deciso in fase di progettazione. L'impostazione del valore statico non permette al modello di adattarsi dinamicamente a variazioni dimensionali o di forma degli oggetti, rendendo l'*NMS* meno efficace in presenza di più oggetti sovrapposti. [35.]

Per quanto riguarda i **vincoli di efficienza** e il **throughput**, nonostante l'efficienza della gestione asincrona attraverso il modificatore *suspend*, l'attuale configurazione presenta un vincolo significativo. Attualmente non sono attivati delegati *GPU/NNAPI* per l'esecuzione che appunto avviene interamente su *CPU*. Questa configurazione, pur garantendo la massima compatibilità, limita drasticamente il numero di elaborazioni per unità di tempo e incrementa la latenza. Il sistema non sfrutta le architetture massivamente parallele tipiche dei moderni processori grafici, rendendolo meno adatto per applicazioni che richiedono elaborazione in tempo reale o su larga scala. [37.]

Queste sfide, tuttavia, offrono l'opportunità di implementare diverse ottimizzazioni. Oltre alle ottimizzazioni intrinseche del modello, come la calibrazione della soglia di confidenza e la funzione *applyNMS()* stessa, è possibile implementare strategie per migliorare ulteriormente le prestazioni e contrastare i vincoli appena esposti. Come è stato introdotto nel capitolo teorico, per migliorare la velocità di inferenza e ridurre le dimensioni del modello, è possibile applicare la **quantizzazione INT8**. Questa tecnica riduce la precisione dei numeri usati per i pesi e le attivazioni, convertendo i valori da virgola mobile a 16 bit a interi a 8 bit. Il procedimento riduce ulteriormente le dimensioni del modello e dell'applicazione, generando efficienza e stabilità. Per calibrare la perdita di precisione, è necessario fornire al *TFLiteConverter* un *representative dataset*. Nell'immagine sottostante viene mostrata una possibile implementazione nella pipeline del caso di studio, la funzione attraverso l'opzione *"TFLITE_BUILTINS_INT8"* utilizza una quantizzazione a int8. [17.]

```python
def convert_to_tflite_int8():
    # TensorFlow Lite Converter API
    converter = tf.lite.TFLiteConverter.from_saved_model(saved_dir)
    # Ottimizzazioni di quantizzazione
    converter.optimizations = [tf.lite.Optimize.DEFAULT]
    # Imposta la funzione per il representative dataset
    converter.representative_dataset = representative_dataset_gen
    # Check sul fatto che il modello possa usare operazioni su interi a 8 bit
    converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
    # Converti il modello
    tflite_model = converter.convert()

    tflite_file_path = "yolov5s-int8.tflite"
    with open(tflite_file_path, "wb") as f:
        f.write(tflite_model)

    print(f"Modello TFLite INT8 creato: '{tflite_file_path}'")
    return tflite_file_path
```

22. funzione di conversione che sfrutta la quantizzazione ad 8 bit.

Nel caso in cui si volesse adottare una strategia del genere nella soluzione, la funzione *convert_to_tflite_int8()* sarebbe sostituita alla funzione che effettua la conversione con 16 bit.

Per superare i vincoli di efficienza sopra illustrati, è stato implementato l'utilizzo di delegati hardware, come *GPUDelegate* o *NnApiDelegate*, per sfruttare gli acceleratori presenti nei dispositivi mobili. L'*API TensorFlow Lite* per Android permette di gestire l'accelerazione hardware direttamente nel codice dell'applicazione. L'obiettivo è creare un interprete che deleghi le operazioni più complesse a un processore specializzato invece di usare solo la CPU.

In ambiente Android Studio per implementare l'utilizzo di *GPUDelegate* è necessario creare un oggetto *Interpreter.Options*, tramite la *new Interpreter.Options()*, che faccia parametro di personalizzazione all'*Interpreter*, oggetto cardine dell'inferenza edge. [19.] Nelle opzioni è necessario aggiungere il delegato appropriato, come mostrato correttamente in figura 23.

```
// Configura le opzioni dell'interprete
Interpreter.Options options = new Interpreter.Options();

// Aggiungi il delegato GPU
GpuDelegate gpuDelegate = new GpuDelegate();
options.addDelegate(gpuDelegate);

// Crea l'interprete TensorFlow Lite e assegna il delegato GPU
tflite = new Interpreter(loadModelFile(assetManager, modelPath), options);
```

23. codice per abilitare l'acceleratore GPU aggiungendo il delegato.

L'attivazione di questi delegati comporta un notevole aumento della velocità di inferenza e un miglioramento del throughput, rendendo il modello più adatto per applicazioni in tempo reale. Spesso, sarebbe una buona prassi verificare la compatibilità del dispositivo prima di aggiungere il delegato, poiché non tutti i dispositivi supportano l'accelerazione hardware per il machine learning. Una volta che l'interprete è stato creato con le opzioni corrette, l'inferenza procede normalmente.

Infine, un'ultima ottimizzazione avanzata che non approfondiremo è la gestione dell'implementazione di un algoritmo di *NMS multi-classe* che considera le interazioni tra le diverse categorie di oggetti, unitamente all'esplorazione di tecniche di calibrazione dinamica per le soglie, potrebbe migliorare significativamente l'accuratezza e la robustezza del sistema di rilevamento, consentendo un adattamento più efficace a scenari complessi. [35.]

3.3 – Anomaly detection con Autoencoder

Precedentemente, è stata esplorata la potenza e la precisione dei modelli pre-addestrati nel campo della rilevazione di oggetti. Tuttavia, le sfide industriali e aziendali spesso richiedono una capacità di rilevamento che trascende la semplice categorizzazione di elementi noti.

In questo paragrafo, verrà presentata l'altra pipeline che tratta il caso di studio. È importante sottolineare ancora come, in entrambi i casi, l'obiettivo finale è la realizzazione di soluzioni efficienti e implementabili su dispositivi edge, grazie alla conversione nel formato *LiteRT*.

L' **Anomaly Detection** si concentra sull'identificazione di deviazioni dalla *"normalità"*. Per la risoluzione di questo problema è stato scelto ed implementato, e verrà presentato, un *autoencoder* convoluzionale. [27.]

La scelta progettuale si allinea perfettamente con l'obiettivo di rilevare deviazioni su un set di dati visuali. Il modello è stato specificamente addestrato per ricostruire fedelmente immagini *"normali"* della categoria *"grid"* del *dataset MVTec AD* di *Paul Bergmann, Michael Fauser, David Sattlegger, e Carsten Steger*, ma potrebbero benissimo essere immagini scattate dal committente dell'applicazione. [38.]

L'obiettivo principale è di istruire il modello per facilitare la ricostruzione fedele di immagini "normali", salvate nella cartella */good* del dataset, rilevando quindi eventuali deviazioni, ovvero anomalie sospette. Queste ultime sono salvate nella cartella */broken* del dataset. Il modello deve essere capace di identificare e quantificare le anomalie come una significativa incapacità di ricostruire l'immagine in ingresso, manifestata da un elevato errore quadratico medio, *MSE*, tra l'immagine originale e quella generata. Tale approccio sfrutta la naturale tendenza dell'autoencoder, addestrato su dati non anomali, a produrre un errore di ricostruzione molto basso per immagini simili a quelle viste in fase di training, mentre lo stesso errore aumenta drasticamente in presenza di imperfezioni. [39.]

3.3.1 – Preparazione del dataset

La preparazione del dataset costituisce spesso un passaggio fondamentale, per implementazioni di questo tipo. A differenza di quanto visto per il modello di rilevamento oggetti, il quale ha beneficiato dell'utilizzo di un modello già pre-addestrato come YOLOv5, la pipeline per l'**Anomaly Detection** ha richiesto un approccio più personalizzato. Questo processo, infatti, dimostra come sia possibile partire da zero, utilizzando immagini di dataset comuni o specifici, per addestrare un modello edge. L'implementazione qui descritta, supportata dallo script *python*, non solo convalida la fattibilità di un tale approccio, ma ne sottolinea l'importanza e la facilità di applicazione in contesti industriali e aziendali. In tali scenari, dove la necessità è di rilevare anomalie su prodotti o componenti, la capacità di scattare autonomamente fotografie di parti sia conformi che difettose e di utilizzarle per creare un modello *LiteRT* semplice ma efficace rappresenta un vantaggio significativo. Questo processo permette di convertire rapidamente i dati in un'applicazione utile e facilmente distribuibile su dispositivi mobili, semplificando la rilevazione di anomalie con una soluzione su misura e pienamente controllabile.
Nell'implementazione, la funzione *prepare_data()* pre-elabora i dataset di immagini, le quali sono state ridimensionate a *128x128 pixel*, normalizzate in un intervallo *[0,1]* e se necessario, convertite in scala di grigi per adattarsi agli input del modello.

```python
# Funzione per caricare e preparare i dati
def prepare_data(data_dir, img_size=(128, 128)):
    images = []
    # Itera sui file della directory specificata
    for filename in os.listdir(data_dir):
        if filename.endswith(".png"):
            filepath = os.path.join(data_dir, filename)
            try:
                # Apri l'immagine, la converte in RGB e ridimensiona
                img = Image.open(filepath).convert('RGB').resize(img_size)
                images.append(np.array(img))
            except Exception as e:
                print(f"Impossibile caricare l'immagine {filename}: {e}")
                continue

    # Normalizza i dati e converte in float32
    images = np.array(images).astype('float32') / 255.0
    return images
```

24. funzione che normalizza tutte le immagini del direttorio preso in esame.

3.3.2 – Addestramento dell'Autoencoder

L'addestramento dell'autoencoder è stato condotto utilizzando esclusivamente un dataset di sole immagini "normali", senza anomalie.

```python
# Utilizza solo le immagini "good" per addestramento
train_dir = 'C:/Users/andre/Downloads/mvtec_anomaly_detection/grid/train/good'
X_train_normal = prepare_data(train_dir)
```

Ovviamente, questa scelta metodologica è cruciale, poiché l'intera logica di rilevamento si basa sulla presupposizione che il modello, non avendo mai visto un'anomalia in fase di training, avrà difficoltà a ricostruirla, manifestando tale incapacità con un elevato errore di ricostruzione. Il modello impara a ricostruire fedelmente queste immagini "normali", minimizzando l'errore tra l'immagine originale e quella ricostruita.

Per quanto riguarda l'architettura, l'ottimizzatore scelto è *Adam*, un algoritmo di ottimizzazione adattivo che si distingue per la sua efficienza e rapidità di convergenza. A differenza di ottimizzatori più semplici, *Adam* aggiusta dinamicamente i tassi di apprendimento per ciascun peso della rete neurale. [40.]

```python
# Crea e compila il modello
autoencoder = create_autoencoder(input_shape)
autoencoder.compile(optimizer='adam', loss='mean_squared_error')
```

Come è stato accennato nel capitolo di teoria, la funzione di perdita spesso utilizzata è

il *Mean Squared Error (MSE)*. Anche in questo caso di studio è utilissima, in quanto questa funzione calcola la media della differenza quadratica tra i valori dei pixel dell'immagine originale e quelli dell'immagine ricostruita.
L'uso di *MSE* è perfettamente allineato con l'obiettivo del modello: minimizzare il valore di questa metrica significa che la rete sta diventando sempre più abile nel riprodurre l'input, una competenza che è alla base del successo del rilevamento di anomalie in un autoencoder.

Il processo di addestramento è chiaro. Nell'implementazione il modello è stato addestrato per cento epoche con una *batch size* di 16. Questi dati sono impostati affinché il modello sia abbastanza performante nella rilevazione di anomalie. In particolare, il numero di epoche è stato ritenuto sufficiente per permettere al modello di convergere verso un errore minimo, utile per il caso di studio, senza incorrere in un eccessivo *overfitting (vedi paragrafo 3.3.5)*.

```
Epoch 85/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 351ms/step - loss: 0.0021 - val_loss: 0.0022
Epoch 86/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 343ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 87/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 351ms/step - loss: 0.0021 - val_loss: 0.0021
Epoch 88/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 344ms/step - loss: 0.0021 - val_loss: 0.0020
Epoch 89/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 346ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 90/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 345ms/step - loss: 0.0021 - val_loss: 0.0020
Epoch 91/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 346ms/step - loss: 0.0020 - val_loss: 0.0020
Epoch 92/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 347ms/step - loss: 0.0020 - val_loss: 0.0021
Epoch 93/100
15/15 ━━━━━━━━━━━━━━━━━━━━ 5s 349ms/step - loss: 0.0020 - val_loss: 0.0022
```

25. screenshot del processo di addestramento di 100 epoche sul dataset di immagini normali.

3.3.3 – Valutazione delle Prestazioni attraverso Soglie di Rilevamento

Il processo di inferenza inizia dopo l'addestramento e la valutazione del modello, una fase fondamentale per stabilire la sua efficacia nel contesto del rilevamento delle anomalie. Come si evince dal codice *Python* in figura 26, la prima fase consiste nell'eseguire previsioni sia su immagini normali che su immagini anomale del set di test, utilizzando il metodo *autoencoder.predict()*. [41.]

```
# Carica i dati di test (immagini normali e anomale)
test_dir_normal = 'C:/Users/andre/Downloads/mvtec_anomaly_detection/grid/test/good'
test_dir_anomalies = 'C:/Users/andre/Downloads/mvtec_anomaly_detection/grid/test/broken'
X_test_normal = prepare_data(test_dir_normal)
X_test_anomalies = prepare_data(test_dir_anomalies)

# Previsione sulle immagini di test
reconstructed_normal = autoencoder.predict(X_test_normal)
reconstructed_anomalies = autoencoder.predict(X_test_anomalies)
```

26. codice che utilizza la autoencoder.predict per eseguire le dovute previsioni sul dataset dopo l'addestramento, per valutarne in futuro la precisione.

Successivamente, viene calcolato il *Mean Squared Error (MSE)* per entrambi i set di immagini, una metrica che quantifica l'errore di ricostruzione del modello. Il codice calcola l'*MSE* per le immagini normali, mentre un calcolo analogo viene eseguito per quelle anomale.

```
# Calcola l'errore di ricostruzione (MSE)
mse_normal = np.mean(np.power(X_test_normal - reconstructed_normal, 2), axis=(1, 2, 3))
mse_anomalies = np.mean(np.power(X_test_anomalies - reconstructed_anomalies, 2), axis=(1, 2, 3))
```

Se l'errore per queste ultime risulta significativamente più elevato, è confermata l'efficacia del modello nel rilevare le deviazioni. L'*MSE* diventa quindi la metrica chiave, la soglia globale per la classificazione finale nel caso di studio.

Questa soglia non è un valore arbitrario, ma viene calcolata in modo statistico: si basa sull'errore di ricostruzione delle immagini "normali" nel set di test. Viene definita come il 95° percentile dell'errore di ricostruzione del set di dati normali, una scelta che si allinea alla regola empirica delle tre *sigma* per identificare valori che si discostano statisticamente dalla norma.

```
# percentile della distribuzione degli errori normali
threshold = np.percentile(mse_normal, 95) # Usa il 95° percentile come soglia
print(f"Soglia di anomalia MSE (95° percentile): {threshold:.6f}")
```

Questa scelta risulta essere più robusta di una semplice media e deviazione standard, in quanto si adatta meglio alla distribuzione effettiva dei dati senza fare assunzioni sulla loro forma.

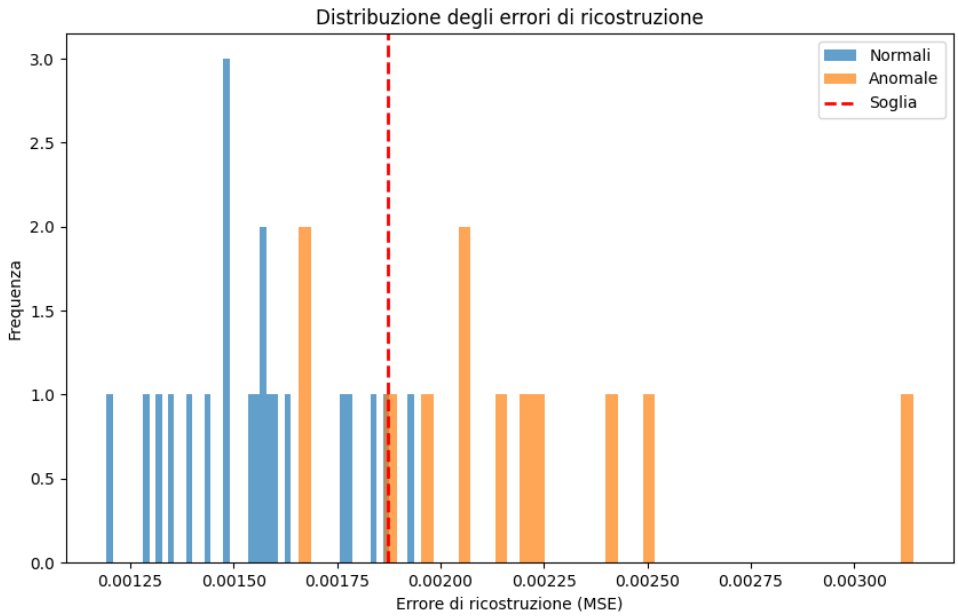
L'obiettivo dell'implementazione è stato raggiungere una situazione stabile in cui la soglia si posizioni correttamente tra gli errori di ricostruzione delle immagini normali e di quelle anomale. Dopo l'addestramento, i valori ottenuti dal test sono: un errore *MSE*

normale di *0,001559*, un errore *MSE* anomalo di *0,002162* e una soglia di anomalia di *0,001872*, che confermano che l'addestramento ha avuto successo.

```
Errore MSE medio per immagini normali: 0.001559
Errore MSE medio per immagini anomale: 0.002162
Soglia di anomalia MSE (95° percentile): 0.001872
```

27. screenshot del terminale che mostra una soluzione corretta della valutazione delle soglie dopo l'addestramento e il test.

Questi dati non devono essere considerati un risultato unico e immutabile. Piuttosto, rappresentano un esempio di successo di un singolo addestramento riuscito. La situazione è inoltre visivamente confermata dal grafico della distribuzione degli errori, che mostra due istogrammi separati con la linea rossa della soglia posizionata in modo quasi ottimale tra i due, confermando ancora una volta che il modello è robusto, e che solamente casi sporadici non rientrano nella classificazione.



28. due istogrammi che mostrano le singole rilevazioni, rapportate alla soglia di anomalia.

3.3.4 – Conversione in LiteRT: Inferenza Android

Al termine del testing, il modello addestrato in Keras è pronto per la conversione nel formato compatto *LiteRT*. Questo passaggio finale, essenziale per l'inferenza su dispositivi edge, lega la fase di addestramento alla sua applicazione pratica. La

conversione, eseguita al solito tramite *TFLiteConverter.from_keras_model()*, ha richiesto in questo specifico caso l'abilitazione delle operazioni *tf.lite.OpsSet.SELECT_TF_OPS* per garantire la piena compatibilità con le operazioni Keras utilizzate nell'architettura. [17.]

Il modello serializzato è così pronto per essere caricato ed eseguito in modo efficiente sull'applicazione Android.

L'applicazione Android, sviluppata in *Kotlin*, gestisce il processo di inferenza in modo efficiente. All'interno dell'architettura dell'app, il flusso di esecuzione è gestito in modo chiaro e sequenziale. La catena di inferenza inizia nell'interfaccia utente *InferenceScreen.kt*, dove un'azione, come il clic su un bottone, innesca la chiamata alla funzione *runInference()* all'interno del *AnomalyViewModel*. Come si evince dal codice del *viewmodel*, la funzione *runInference()* avvia il calcolo in un thread in background per evitare di bloccare l'interfaccia utente.

All'interno di questo *thread*, la funzione chiama il metodo *detect()* di *TfLiteAnomalyDetector*. È in questo passaggio che l'interprete *TFLite* calcola la soglia *MSE*. Il valore *MSE* viene quindi restituito al v*iewmodel*, che ne valuta la magnitudo confrontandolo con la soglia di anomalia dell'utente.

```
Log.d( tag = TAG,  msg = "Avviato il rilevamento delle anomalie.")
anomalyViewModel.runInference { isAnomaly ->
    val msg = when (isAnomaly) {
        null -> "Errore rilevamento: ${anomalyViewModel.lastError ?: "sconosciuto"}" ^runInference
        true -> "Anomalia rilevata (MSE: ${"%.6f".format( ...args = anomalyViewModel.mseValue)})" ^runInference
        false -> "Nessuna anomalia (MSE: ${"%.6f".format( ...args = anomalyViewModel.mseValue)})" ^runInference
    }
    Log.i( tag = TAG,  msg = "Risultato rilevamento anomalie: $msg")
    Toast.makeText(context,  text = msg,  duration = Toast.LENGTH_LONG).show()
    // Mostra popup statistiche anomalia al termine
    showAnomalyStatsDialog = true
}
```

29. parte del codice del viewmodel, che lancia un thread per fare inferenza.

È importante sottolineare che, sebbene la soglia possa essere regolata attraverso uno slider, offrendo flessibilità in base a specifiche esigenze operative, è sempre bene utilizzare come punto di partenza il valore a default trovato in fase di addestramento, ovvero *0.001872*. Questo valore rappresenta lo spartiacque ottimale tra le immagini normali e quelle anomale, garantendo un equilibrio tra sensibilità e precisione. La decisione finale di anomalia rilevata o meno viene poi esposta nell'interfaccia utente.

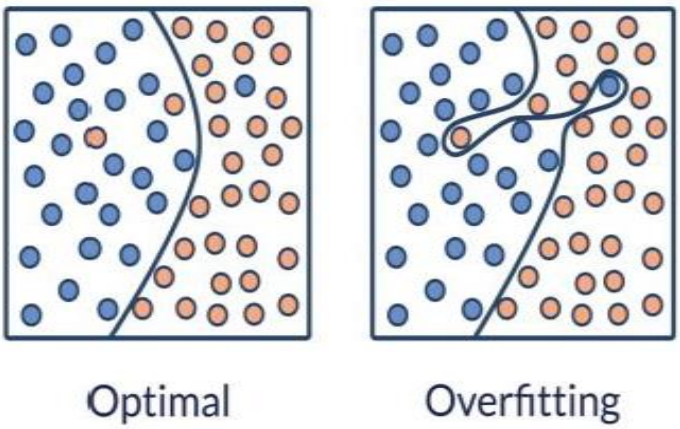
3.3.5 – Limiti ed Ottimizzazioni

In questo paragrafo sono analizzate eventuali problematiche ricorrenti in queste tipologie di modelli *LiteRT* di rilevazione anomalie.

Criticità legate all'Addestramento e alla Natura delle Anomalie: la quantità e la varietà ridotta di immagini di addestramento "normali" impediscono al modello di apprendere una rappresentazione robusta, limitando la sua capacità di discernere efficacemente le anomalie. Inoltre, un numero insufficiente di epoche di addestramento, come le 50 iniziali poi aumentata a 100, potrebbe non permettere al modello di convergere a un minimo di errore accettabile, aumentando anche il numero di falsi positivi.

Infine, se le immagini con difetti sono troppo simili alle variazioni "normali", l'autoencoder non sarà in grado di differenziare con un errore di ricostruzione significativo, compromettendo la sensibilità e robustezza del sistema.

Il Problema dell Overfitting: quando si aumenta il numero di epoche per l'addestramento, un punto critico da monitorare è l'*overfitting*. Se il modello viene addestrato per troppe epoche, tende a memorizzare il set di dati di addestramento invece di imparare a generalizzare. In questo caso, l'errore sui dati di addestramento continuerà a diminuire, mentre l'errore sui dati di validazione inizierà ad aumentare. Il modello non riesce più a distinguere le caratteristiche importanti da quelle casuali. In sintesi, un numero maggiore di epoche non significa sempre più precisione.

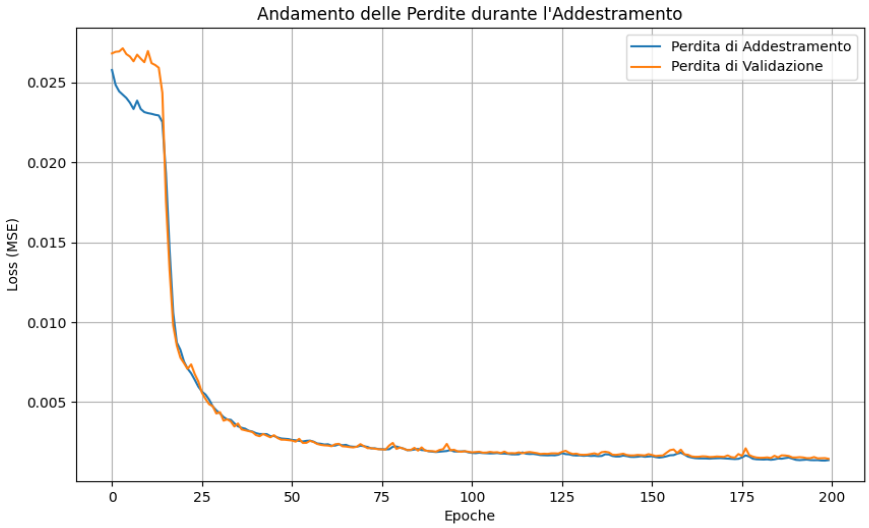


30. spiegazione visiva semplificata del fenomeno "overfitting", il modello non generalizza.

È fondamentale monitorare sia l'errore di addestramento che quello di validazione per trovare il punto ottimale, un equilibrio in cui il modello è ben addestrato ma non sovra-adattato. Aumentare le epoche da *50* a *100*, come nell'esempio dell'autoencoder, può aiutare un modello "più debole" a convergere senza cadere nell'*overfitting*. Se si nota che l'errore di validazione comincia a salire, è il momento di fermarsi.

Il modo più efficace per monitorare questo fenomeno è visualizzare un grafico che mostra l'andamento di queste due perdite nel corso delle epoche. Per interpretare il grafico, è necessario osservare le due curve; se la curva della perdita di validazione continua a scendere insieme alla perdita di addestramento, l'addestramento sta procedendo bene. Se, invece, la curva della perdita di validazione inizia a salire mentre la perdita di addestramento continua a scendere, il modello sta iniziando a sovra-adattarsi. In quel caso, dovresti considerare di ridurre il numero di epoche. [42.]

Questo grafico fornisce una segnalazione visiva che aiuta a capire quando fermare l'addestramento. Nell'implementazione, per fare un test, il modello è stato allenato il modello per addirittura *200* epoche, e comunque dal grafico non si notano particolari sovra-adattamenti.
Come è stato descritto in fase di implementazione, per monitorare e mitigare il rischio di *overfitting*, il *10%* dei dati di addestramento è stato segregato e utilizzato come set di validazione. Le due curve vanno di pari passo; si evince quindi che grazie all'utilizzo di questa contromisura il modello risulta essere particolarmente robusto.



31. grafico che mostra la curva di apprendimento e quella di perdita di valutazione

Nel caso in cui fosse necessario un modello ancora più performante o si dovesse addestrare il modello per più di *200* epoche, una tecnica molto efficace di ottimizzazione, in particolare di preprocessing, è l'applicazione della *data augmentation* sul set di addestramento. Questa tecnica non è stata inclusa nell'implementazione originale, ma rappresenta una pratica comune per migliorare le prestazioni del modello. L'obiettivo principale della tecnica è aumentare la dimensione e la variabilità del set di dati, applicando trasformazioni casuali come rotazioni, zoom, riflessioni o piccole variazioni di luminosità alle immagini "normali". [43.]

Vincoli di Efficienza e Throughput: anche in questa pipeline, sul fronte dell'efficienza, ci sono dei limiti. Attualmente l'esecuzione è *CPU-only* e non sfrutta delegati hardware come *GPU* o *NNAPI*, il che ha un impatto su latenza e throughput, specialmente su dispositivi mobili.

Per ottimizzare le prestazioni, sarebbe necessario attivare questi delegati. L'app utilizza già alcune ottimizzazioni come l'esecuzione su *Dispatchers.Default* per non bloccare il thread *UI* e l'uso di buffer diretti per minimizzare le allocazioni di memoria. [19.]

3.4 – Discussione e Commento

Per concludere, questo caso di studio implementa una soluzione di visione artificiale basata su modelli *LiteRT*. L'applicazione sviluppata deve essere una prova delle potenzialità tecnologiche del *runtime*; il caso preso in esame è solamente un esempio.

Ad oggi, l'adozione di modelli *LiteRT* e l'applicazione di tecniche di ottimizzazione non sono più un'opzione, ma un'esigenza per chiunque voglia sviluppare soluzioni di visione artificiale avanzate, affidabili e pronte per il mercato.

Conclusioni e Direzioni Future

La trattazione ha esplorato e dimostrato il potenziale dell'*AI on the Edge*, sottolineando come le potenzialità di questo approccio aprono la strada a innovazioni future in settori come la manutenzione predittiva, la logistica e il controllo qualità automatizzato. I risultati ottenuti hanno confermato che l'ottimizzazione di modelli complessi permette di eseguire operazioni di inferenza direttamente su dispositivi *edge*, mantenendo un'elevata precisione e affidabilità rispondendo alle crescenti esigenze di efficienza e sicurezza delle nuove generazioni di applicazioni intelligenti e decentralizzate. È per questi motivi che si è scelto fin da subito di operare interamente con inferenze *on-device*. Questo approccio sposta il paradigma dall'infrastruttura centralizzata al *computing on the edge*, offrendo chiari vantaggi come la riduzione della latenza, una maggiore privacy dei dati sensibili e una migliore efficienza rispetto alle soluzioni basate su cloud.

L'adozione su larga scala dell'AI on the Edge è destinata a ridisegnare i processi produttivi e logistici, soprattutto in ambito industriale e aziendale, rendendoli più agili, sicuri e sostenibili. L'elaborazione locale dei dati consente di identificare potenziali guasti o inefficienze prima che si verifichino, inviando un allarme istantaneo e riducendo drasticamente i tempi di inattività e i costi di manutenzione non pianificata.

E' doveroso sottolineare che, quindi, il futuro appare promettente: l'evoluzione di *runtime* come **LiteRT** favorirà l'interoperabilità tra i vari framework, rendendo la distribuzione dei modelli AI sempre più fluida.
L'obiettivo è chiaro: la tecnologia si sta integrando come componente nativo e indispensabile per l'elaborazione dei dati.
Questo, molto probabilmente, permetterà di sbloccare nuove opportunità di business, creando sistemi intelligenti che sono non solo più veloci ed efficienti, ma anche più robusti e sicuri in ambiti difficoltosi, come quello *Android*.

Bibliografia

Tecnologie Edge AI

1. Documentazione LiteRT: Google. LiteRT documentation. Disponibile su: *https://ai.google.dev/edge/litert.*
2. Bnova. "Modelli di machine learning: cosa sono e come scegliere il migliore.". Disponibile su: https://www.bnova.it/data-science/machine-learning-modelli/.
3. IBM. "Edge AI: Che cos'è l'Edge AI". Disponibile su: *https://www.ibm.com/it-it/topics/edge-ai*.
4. "Il Training dei Modelli di Intelligenza Artificiale: Dati, Tempistiche e Implicazioni.". Disponibile su: *https://mindthechartdatabase.mindthechart.io/il-training-dei-modelli-di-intelligenza-artificiale-dati-tempistiche-e-implicazioni/*.
5. Wikipedia. Fine-tuning. Disponibile su: *https://it.wikipedia.org/wiki/Fine-tuning_(deep_learning)*.
6. Astera. Pre-processing. Disponibile su: *https://www.astera.com/it/type/blog/data-preprocessing*.
7. Wikipedia. Postprocessing. Disponibile su: *https://en.wikipedia.org/wiki/Postprocessing*.
8. Salvino Fidacaro. Inferenza nei modelli di ML. Disponibile su: *https://www.fidacaro.com/*.
9. Microsoft Azure. Cloud Computing. Disponibile su: *https://azure.microsoft.com/*.
10. Mao, Y., et al. "A survey on edge computing and its applications." IEEE Access, vol. 6, 2018, pp. 6427-6441. Disponibile su: *https://ieeexplore.ieee.org/*.
11. Shi, W., et al. "Edge computing: The road to ubiquitous intelligence." Proceedings of the IEEE, vol. 107, no. 8, 2019, pp. 1474-1488. Disponibile su: *https://arxiv.org*.
12. Google, "Federated Learning: Collaborative Machine Learning without Centralized Training Data.". Disponibile su: *https://ai.googleblog.com/2017/04/federated-learning-collaborative.html*.
13. OneTrust. *X Privacy*. Disponibile su: *https://www.onetrust.com/*.
14. V. I. Iorga, "Privacy-preserving machine learning: a survey." arXiv preprint arXiv:2009.07168, 2020. Disponibile su: *https://arxiv.org/abs/2108.04417*.
15. Global Sustainability Barometer. Microsoft and Kyndryl. Microsoft Global Sustainability Barometer. Disponibile su: *https://sustainabilitymag.com*.
16. Real Python. TensorFlow vs PyTorch. Disponibile su: *https://realpython.com/pytorch-vs-tensorflow.*
17. Google. TensorFlow Lite documentation. Disponibile su: *https://www.tensorflow.org/lite/*.
18. GitHub di LiteRT, sviluppatori LiteRT. Disponibile su: *https://github.com/google-ai-edge/litert*.

19. Delegati LiteRT. Documentazione LiteRT: Google. Disponibile su: *https://ai.google.dev/edge/litert/performance/delegates*.
20. TuttoDigitale. Pianificazione Accelerazione GPU. Disponibile su: *https://tuttodigitale.net/tec/come-abilitare-la-pianificazione-gpu-con-accelerazione-hardware*.
21. Supporto multi-framework. Documentazione LiteRT: Google. Disponibile su: *https://ai.google.dev/edge/litert/models*.
22. TFLiteConverter. Documentazione LiteRT: Google. Disponibile su: *https://ai.google.dev/edge/api/tflite/python/tf/lite/TFLiteConverter*.
23. Milvus: L'articolo “How does edge AI enable offline machine learning applications”. Disponibile su: *https://blog.milvus.io/ai-quick-reference*.
24. Jacob, B., Kligys, S., Chen, B., et al. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. Disponibile su: *https://arxiv.org/abs*.
25. Han, S., Mao, H., & Dally, W. J. (2015). Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. Disponibile su: *https://openreview.net*.
26. Wikipedia. Autoencoder. Disponibile su: *https://en.wikipedia.org/wiki/Autoencoder*.
27. ScienzeDirect. A comprehensive study of auto-encoders for anomaly detection: Efficiency and trade-offs. Disponibile su: *https://www.sciencedirect.com/science/article*.
28. PyTorch. PyTorch Mobile. Disponibile su: *https://pytorch.org/mobile*.
29. Microsoft. ONNX Runtime. Disponibile su: *https://onnxruntime.ai/*.
30. Apple. Core ML. Disponibile su: *https://developer.apple.com/documentation/coreml*.
31. A Comparative Survey of PyTorch vs TensorFlow for Deep Learning: Usability, Performance, and Deployment Trade-offs." arXiv preprint arXiv:2508.04035, 2025. Disponibile su: *https://arxiv.org/abs/2508.04035*.

Modelli, Dataset e Strumenti per il Caso di Studio

32. Repository Ufficiale di YOLOv5: Ultralytics. YOLOv5 GitHub repository. Disponibile su: *https://github.com/ultralytics/yolov5*.
33. Script di Esportazione YOLOv5: Ultralytics. *export.py*. Disponibile su: *https://github.com/ultralytics/yolov5/blob/master/export.py*.
34. Documentazione Ultralytics (Esportazione): Exportation to TFLite, ONNX, CoreML, TensorRT. Disponibile su: *https://docs.ultralytics.com/yolov5/tutorials/model-export/*.
35. J. Redmon, et al. "You Only Look Once: Unified, Real-Time Object Detection." Proceedings of the IEEE Conference on Computer Vision, 2016, pp. 779-788. Disponibile su: *https://ieeexplore.ieee.org/*.
36. Kamar, R., & Jain, S. "Scalability of YOLOv5 and Tensorflow Lite." ScholarWorks, 2025.

37. M. Sandler, et al. "MobileNetV2: Inverted Residuals and Linear Bottlenecks." Proceedings of the IEEE Conference on Computer Vision, 2018, pp. 4510-4520. Disponibile su: *https://ieeexplore.ieee.org/*.
38. Dataset per Rilevamento di Anomalie: Bergmann, P., Fauser, M., Sattlegger, D., & Steger, C. (2019). A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection. IEEE Conference on Computer Vision. Disponibile su: *https://www.mvtec.com/company/research/datasets/mvtec-ad*.
39. IBM. Cos'è un autoencoder?. Disponibile su: *https://www.ibm.com/it-it/topics/autoencoders*.
40. Kingma, D. P., & Ba, J. "Adam: A Method for Stochastic Optimization." Disponibile su: *https://arxiv.org/pdf*.
41. TensorFlow. Intro to Autoencoders. Disponibile su: *https://www.tensorflow.org/tutorials/generative/autoencoder*
42. IBM. Overfitting. Disponibile su: *https://www.ibm.com/it-it/think/topics/overfitting*
43. Wikipedia. Data augmentation. Disponibile su: *https://en.wikipedia.org/wiki/Data_augmentation*.